Generic eGRM Platform

System Specifications — Consolidated Reference Document

Configurable, multi-client electronic Grievance Redress Mechanism
Derived from the KISIP implementation and KUSP2 requirements

Version 1.0 • 12 June 2026

Contents

01 — Overview, Goals and Architecture Principles

1. Purpose

This specification defines a **generic, multi-client electronic Grievance Redress Mechanism (eGRM) platform**. The platform is a configurable product: a single codebase that can be deployed and configured for different clients (government programmes, donor-funded projects, corporations) with materially different grievance processes — without code changes.

The specification is derived from three knowledge sources:

Source	Contribution
KISIP eGRM (`plus-admin` , in production)	Field-proven mechanics (SMS tracking links, notification audit, confirmation-before-closure) and an inventory of design defects to avoid (client-side state machine, hardcoded hierarchy/roles/SLAs, unauthenticated write endpoints)
KUSP2 eGRM procurement set (BRD, ToR + SRS, wireframe report, technical specs, RFQ — Feb 2026)	The requirements baseline: 136 traceable requirements, configurable helpdesk-style architecture (help topics, SLA plans, routing filters), standardized intake dataset, SEA/SH module, NFRs
GRM best-practice corpus (World Bank ESS10 doctrine, NAVCDP, TAP, Cassava, PSCK, IFC/CAO)	Process doctrine: resolve at lowest level, acknowledgement/response/resolution SLAs, appeal windows, satisfaction capture, standard KPIs, anonymous and survivor-centred handling

2. Product vision

> One platform, many programmes. Every programme gets its own grievance taxonomy, administrative hierarchy, workflow, SLAs, channels, languages, branding and reporting — all defined as configuration, owned by the client, and changeable by an administrator without redevelopment.

2.1 What "generic" means here (hard requirements)

1. **No hardcoded administrative hierarchy.** KISIP uses settlement→county→national; KUSP2 uses municipality→county→national; another client may use site→region→HQ or a 2-level structure. The hierarchy is tenant data, of arbitrary depth.
2. **No hardcoded workflow.** Statuses, transitions, who may perform them, required evidence, SLA per stage, escalation triggers, closure/confirmation rules and appeal windows are configuration.
3. **No hardcoded taxonomy.** Case categories ("help topics"), sensitivity classes (GBV/SEA-SH, corruption, …), priorities and expected-outcome lists are configuration.
4. **No hardcoded channels.** Web, assisted intake, email, SMS, USSD, hotline/IVR, API — each is a module enabled/disabled and configured per tenant.
5. **No hardcoded text or branding.** All complainant-facing strings are templates with variables, multi-language, owned by the tenant. Reference-number format, portal theme, logos, legal pages are tenant config.
6. **No hardcoded notification logic.** Recipient selection, channel choice and message content are declarative rules evaluated by a notification engine.

2.2 What is fixed (the platform invariants)

Some things are deliberately *not* configurable, because they are correctness or safety properties:

- Server-side enforcement of the configured workflow (clients cannot bypass it).
- Atomicity: a case action (status change + log entry + attachments + notifications queued) commits in one transaction.
- Append-only audit trail for every state-changing operation, including reads of sensitive cases.
- PII handling rules: field-level encryption at rest, redaction by role, consent capture, retention enforcement.
- Authentication on every endpoint except the explicitly designated public surface (intake, status lookup, knowledge base), which is rate-limited and abuse-protected.
- A case always has: a tenant, a unique reference, a category, a jurisdiction, a status from the tenant's configured status set, and a complete history.

3. Tenancy and deployment models

The platform MUST support both deployment models below from the same codebase; the choice is per contract:

Model	Description	When
D1 — Dedicated instance per client	One deployment, one database per client (KISIP-style; satisfies strict data-residency / government-hosted requirements such as KUSP2 Appendix C Option A)	Government clients, data-sovereignty constraints
D2 — Shared multi-tenant SaaS	One deployment, row-level isolation by `tenant_id` (with optional schema-per-tenant), shared infrastructure	SMEs, pilots, vendor-hosted offering (KUSP2 Option B, ≥99.5% availability)

Design consequences:

- Every table that holds tenant data carries `tenant_id`; all queries are tenant-scoped at the data-access layer (enforced centrally, e.g. PostgreSQL row-level security), even in D1 (where there is exactly one tenant row). This keeps D1→D2 migration trivial.
- Configuration is stored in the database (not env files) so it is exportable, versioned, and editable through the admin console. Environment variables hold only infrastructure secrets.
- A **tenant configuration export/import** (single signed bundle) supports promotion across environments (dev → staging → prod) and client onboarding from a template.

4. High-level architecture

```
flowchart LR
  subgraph Channels
    WEB[Public portal]
    USSD[USSD gateway]
    HOT[Hotline / assisted intake]
    MAIL[Email in/out]
    API[Partner API]
  end
  subgraph Core["Core platform (per tenant config)"]
    INTAKE[Intake service<br/>forms, validation, dedupe]
    WF[Workflow engine<br/>statuses, transitions, guards]
    SLA[SLA engine<br/>calendars, timers, escalation]
    NOTIF[Notification engine<br/>rules, templates, channels]
    CASE[(Case store<br/>+ audit + attachments)]
    CFG[(Configuration registry)]
  end
  subgraph Consoles
    STAFF[Staff console]
    ADMIN[Admin console]
    REPORT[Dashboards & reports]
  end
  Channels --> INTAKE --> CASE
  STAFF --> WF
  WF --> CASE
  SLA --> WF
  WF --> NOTIF
  CFG -.drives.-> INTAKE & WF & SLA & NOTIF & STAFF & REPORT
  CASE --> REPORT
```

Components (logical; deployable as a modular monolith initially):

Component	Responsibility
Configuration registry	Versioned, validated tenant configuration: hierarchy, taxonomy, workflow, SLA plans, forms, templates, roles, channels, branding. See spec 02.
Intake service	Channel adapters normalize submissions into the standardized intake dataset; validation, duplicate detection, consent capture, reference generation. See spec 05.
Workflow engine	Server-side state machine instantiated from config; executes case actions atomically with guards. See spec 04.
SLA engine	Computes due dates from SLA plans + working calendars; emits overdue/at-risk events; runs auto-escalation rules. See spec 04.
Notification engine	Declarative event→recipient→template→channel rules; queued delivery with per-message audit and per-channel kill switches. See spec 06.
Case store	Cases, parties, threads, tasks, attachments, referrals, appeals, satisfaction; append-only audit. See spec 03.
Consoles	Public portal, staff console, admin console, dashboards. Behavioral catalogue follows the KUSP2 wireframe report (145 screens) adapted to config-driven rendering.

5. Design principles

1. **Configuration over code; convention over configuration.** Ship sensible defaults (a reference workflow, standard categories, standard KPIs) so a new tenant is usable in hours; everything is overridable.
2. **The server owns the rules.** The frontend renders what the config and the engine allow; it never decides what is allowed. (Direct lesson from KISIP, where transitions, SLA arithmetic and escalation targets live in Vue components — including a days-vs-milliseconds bug that makes every case instantly overdue.)
3. **Everything observable.** Every action, notification, config change and sensitive-data access is auditable. Disabled notifications are still logged with the reason (a good KISIP pattern to keep).
4. **Lowest-level resolution first.** The default workflow encodes the GRM doctrine: cases start at the lowest configured jurisdiction tier, escalate upward on time/severity/dissatisfaction, and may be returned downward.
5. **Safety by default for sensitive cases.** Sensitivity classes restrict visibility, suppress PII in notifications, force aggregate-only reporting, and route to designated roles — per configured policy.
6. **Complainant-centred closure.** Resolution is not closure: the loop closes with complainant feedback (satisfaction), an appeal window, and (optionally) higher-level confirmation — all configurable per tenant.
7. **No lock-in.** Full data + configuration export in open formats at any time (KUSP2 NFR-17); documented APIs (NFR-16).

6. Scope

In scope

- Multi-channel grievance intake, case management, workflow/SLA/escalation, appeals, referrals, tasks, notifications, knowledge base, reporting/dashboards, admin configuration, audit, public portal with status tracking, sensitive-case module, multi-language.

In scope — opt-in modules (default off)

- Chatbot intake and AI assistance (auto-categorization, sensitivity detection, semantic dedupe, summarization, translation, draft responses, KB answer assist) — governed, per-tenant opt-in (spec 05 §7). KUSP2 prohibits chatbot intake (FR-PUB-15), so its profile keeps the flag off.

Out of scope (v1)

- Offline-first mobile sync (low-bandwidth web + assisted intake + USSD cover the need).
- Payments/compensation disbursement (record outcomes only; integrate via API if needed).
- Full document-management system (attachments with access control only).

7. Reading order

Doc	Contents
`02-configuration-model.md`	The per-client configuration registry — the heart of genericity
`03-domain-model.md`	Entities and data model
`04-workflow-engine.md`	Lifecycle, SLA, escalation, appeals, closure
`05-intake-and-channels.md`	Channels, forms, standardized intake dataset
`06-notifications.md`	Notification rules, templates, delivery
`07-security-access-control.md`	RBAC, jurisdiction scoping, sensitive cases, PII
`08-reporting-kpis.md`	Dashboards, KPIs, exports
`09-api-integrations.md`	API surface, webhooks, SSO, gateways
`10-requirements-catalogue.md`	Traceable requirements with priorities and source mapping
`11-tenant-profiles.md`	Worked examples: KISIP and KUSP2 expressed as configuration

02 — Configuration Model (Per-Client Configurability)

This document defines **what is configurable per tenant, where the configuration lives, and how it is governed**. It is the core of the platform's genericity: onboarding a new client means authoring a tenant profile, not writing code.

1. Configuration registry

1.1 Storage and structure

- All tenant configuration is stored in the database in a **configuration registry**, organized as named, typed **config domains** (see §2). Environment variables carry only infrastructure secrets (DB credentials, gateway API keys).
- Every config object is **versioned**: changes create a new version with `changed_by`, `changed_at`, change note, and a diff. Administrators can view history and **roll back** (KUSP2 §4.6).
- Config changes are **validated before activation** (schema validation + referential checks + workflow reachability checks, see §3).
- The full tenant profile is **exportable/importable as one signed bundle** (JSON/YAML) for environment promotion and client onboarding from templates.

1.2 Scoping and precedence

Configuration resolves in order, most specific wins:

```
platform default  →  tenant  →  jurisdiction unit (optional)  →  category/case-type (optional)
```

Example: the tenant sets resolution SLA = 30 days; the "Corruption/Fraud" category overrides to 45 days; a specific county overrides acknowledgement templates with its own letterhead.

2. Configuration domains

CD-01 Tenant identity & branding

Item	Notes
Tenant name, legal name, programme metadata	Displayed in portal/PDFs
Logos, color theme, custom CSS, login background	KUSP2 Theme screens
Portal content pages (about, privacy notice, accessibility, offline notice)	CMS-style, multi-language, versioned (privacy notice versions are referenced by consent records)
Default + enabled locales	e.g. `en`, `sw`; all complainant-facing text must have a value per enabled locale
Timezone & date formats	UTC storage, local display
Free-of-charge & non-retaliation statements	Mandatory display items (KUSP2 FR-PUB-12/14); text editable, presence enforced

CD-02 Administrative hierarchy (jurisdictions)

Item	Notes
Level definitions: ordered list, arbitrary depth	e.g. KISIP: `settlement < county < national`; KUSP2: `municipality < county < national`; corporate: `site < region < HQ`
Unit tree: instances of each level	e.g. 45 counties, 79 municipalities; importable via CSV
Per-level flags	`is_intake_default` (where new cases start), `is_confirmation_authority` (which level confirms closures), `can_be_assigned`
Geo metadata (optional)	Coordinates/boundaries for map dashboards

The hierarchy drives: case routing, escalation direction (always to the next-higher level unless a rule overrides), user scoping, and report disaggregation.

CD-03 Case taxonomy

Item	Notes
Case types	e.g. Grievance, Feedback/Suggestion, Information Request, Safety Incident. Each case type binds: an intake form, a workflow, SLA plan defaults, numbering scheme

Item	Notes
Categories (help topics), hierarchical, multi-select	e.g. Environmental, Social, GBV/SEAH, Corruption/Fraud, Project Implementation, Land/Compensation, … Each category can bind: default department/queue, SLA overrides, sensitivity class, routing rules
Sensitivity classes	Named policies (e.g. `standard`, `gbv_seah`, `corruption`) — each defines visibility restrictions, notification redaction, reporting mode, designated-role routing (spec 07)
Priorities	Ordered list (e.g. Low/Normal/High/Emergency) with SLA multipliers or per-priority SLA plans
Expected outcomes	Multi-select list (information, corrective action, compensation, accountability, other)
Closure reason codes	resolved, referred-out, insufficient information, duplicate/merged, withdrawn, …
Controlled lists	Arbitrary admin-defined lists usable in custom form fields (KUSP2 Lists)

CD-04 Workflow definitions (per case type)

Detailed in spec 04. Configurable elements:

Item	Notes
Status set	Names, localized labels, semantic tag (`open`, `in_progress`, `resolved`, `closed`, `rejected`, `on_hold`, `appeal`) — semantic tags let reports work across tenants with different status names
Transitions	from→to pairs; each with: allowed roles, allowed jurisdiction levels, required fields, required attachment kinds (e.g. signed resolution form), required note, side effects (set assignee, move level, start/stop SLA clock)
Initial status rules	e.g. default `Received`; category- or flag-driven overrides (KISIP: in-court cases start as `In Court`; GBV starts at national level)
Closure policy	closure checklist fields, supervisory-approval threshold (priority/sensitivity/escalated), confirmation step (which level confirms which levels' resolutions), complainant-satisfaction capture on/off
Appeal policy	appeal window (days after resolution notice), who may appeal (complainant/representative), where the appeal routes (next level / committee), max appeal rounds
Reopen policy	who, until when, resulting status

CD-05 SLA plans & calendars

Item	Notes
SLA plans: named sets of targets	`acknowledge_within`, `first_response_within`, `resolve_within`, optional per-status stage durations; units in working or calendar time
Calendars: working hours, weekends, holiday lists	per tenant, optionally per jurisdiction unit
Reminder ladder	e.g. notify assignee at T-2 days, daily after breach
Escalation rules	trigger (SLA breach, status age, priority, dissatisfaction, manual), action (move level, reassign, notify role X, raise priority)
Binding	SLA plan per case type, overridable per category and per priority

CD-06 Intake forms & data standards (per case type / channel)

Item	Notes
Base standardized intake dataset (spec 05 §3)	Field-by-field: enabled, required/optional/conditional, label per locale, help text
Custom fields	Typed (text, number, date, select-from-list, multi-select, attachment), with conditions (show if X)
Channel minimums	Per channel, the minimal field subset (e.g. USSD: location + summary), with `requires_completion` flag for staff follow-up
Anonymity policy	allowed / not allowed per case type; behavior of follow-up when anonymous
Representative submission	enabled + consent-of-affected-person requirement
Attachment policy	max size, count, allowed types, malware-scan toggle
Duplicate detection rule	fields considered + match strategy + behavior (warn / link / block)
Consent configuration	consent text (versioned), required checkboxes
CAPTCHA / abuse controls	on/off, provider, thresholds

CD-07 Reference numbering

Item	Notes
Pattern per case type	e.g. `GRM-{YYYY}-{seq:4}` (KISIP), `{unit_code}-{seq:6}`; tokens: year, tenant code, unit code, sequence with scope (global/yearly/per-unit)
Public verifier	whether status lookup needs reference + phone/email verifier (recommended default: yes)

CD-08 Channels

Per channel: enabled flag + channel-specific settings (spec 05). Web portal modules (knowledge base, status check, registered accounts), USSD menu tree + gateway, hotline scripts + call-center metadata fields, email mailboxes (in/out), SMS gateway provider profile, partner API keys.

CD-09 Notification rules & templates

Detailed in spec 06. Per tenant: event subscriptions, recipient selectors, templates per locale and channel, per-module kill switches (with audit), quiet hours, sender identities (SMS shortcode, from-addresses).

CD-10 Org structure, roles & access

Item	Notes
Departments & teams	routing containers with access lists
Roles: named permission sets	built from the fixed platform permission catalogue (spec 07 §2); tenants can define any number of roles
Jurisdiction scoping rules	role assignment = role × jurisdiction unit(s) × optional department
Sensitive-case designations	which roles handle which sensitivity classes
Authentication policy	password policy, MFA required for which roles, SSO (OIDC/SAML) config, session timeout, IP allowlists

CD-11 Committees (optional module)

Item	Notes
Committee definitions per jurisdiction level	e.g. Settlement GRC, County GMC, National committee
Membership rosters with terms	links to user accounts where members are staff
Workflow bindings	transitions that require a committee decision record (minutes, decision, date, attendees)

CD-12 External referral directory

Item	Notes
Institutions (e.g. EACC, NEMA, Ombudsman/CAJ, GBV service providers)	name, type, contact, jurisdictions covered
Referral policy per category	e.g. criminal matters → police; out-of-scope routing guidance text

CD-13 Reporting & retention

Item	Notes
KPI targets	target values per standard KPI (e.g. resolution ≤30 days)
Public transparency page	on/off; which aggregate stats are published
Scheduled report definitions	recipients, cadence, format
Retention policies	per case type/sensitivity: retention period, then anonymize/archive/delete; legal-hold flag support
Export field policies	which roles may export which fields (PII excluded by default)

CD-15 Dashboards & analytics (admin-built dashboards)

Modeled on the proven `plus-admin` dynamic-dashboard system (dashboard → section → card/chart rows resolved at runtime), hardened with a semantic layer (spec 08 §2–3):

Item	Notes
Dashboard definitions	title, icon, audience (roles/levels), `is_main`, `is_public` (transparency page), layout

Item	Notes
Sections	titled, icon/color groups of widgets within a dashboard
Widgets	KPI cards and charts; each declares: dataset (from the curated semantic layer), measure + aggregation, group-by dimensions, filters, chart kind, time dimension, drill-down target, caption/source note
Chart kinds	named catalogue (kpi_card, bar, stacked_bar, line, multi_line, pie/donut, treemap, map, table, pyramid) — extensible renderer registry
Targets/thresholds	per widget: target value, warning bands, conditional coloring
Audience & scoping	widgets render only for permitted roles; data auto-filtered to the viewer's jurisdiction scope and sensitivity clearance

CD-16 Chatbot & AI assistance (opt-in)

Detailed in spec 05 §7. Default-off; each capability independently switchable:

Item	Notes
AI provider profiles	OpenAI / Azure OpenAI / xAI / self-hosted model; keys in vault; data-residency + no-training flags declared per profile (KISIP precedent: `document_ai` service with OpenAI/xAI keys)
Chatbot channel	surfaces (web widget, WhatsApp), persona/tone text, locales, allowed intents (intake, status check, KB FAQ), escalation-to-human rules, automated-agent disclosure text (presence enforced)
Staff/triage capabilities	per-capability flags + model choice: auto-categorization, sensitivity detection, semantic dedupe, summarization, translation, draft responses, KB answer assist
Safety policy	PII redaction before external calls, sensitive-class processing policy, confidence thresholds, per-capability kill switches (audited)

Platform-level AI governance rules (human-in-the-loop, audit of every suggestion, sensitive fail-safe) are **not** configurable — see spec 05 §7.3.

CD-14 Feature flags (module activation)

Coarse switches for whole modules so light tenants stay simple: knowledge base, tasks, committees, appeals, satisfaction survey, public transparency page, registered complainant accounts, organizations directory, USSD, hotline, public API, custom dashboards, chatbot intake, AI assistance.

3. Configuration governance

Rule	Specification
Validation	A config version cannot activate unless: schema-valid; all references resolve (statuses in transitions exist, roles exist, templates exist for all enabled locales of all subscribed events); the workflow graph has a path from every initial status to a `closed`-tagged status; every SLA plan referenced exists
Dry-run / simulation	Admin can run a synthetic case through a draft workflow (state machine walk) before activation
Staged activation	New config versions activate atomically; in-flight cases either continue on the version they started with (default) or are migrated by an explicit, logged mapping (old status → new status) chosen by the admin
Audit	Every config change is an audit event (actor, domain, diff, version)
Permissions	Config administration is a separate permission family from case handling (separation of duties; KUSP2 FR-ADM-01)
Templates library	The platform ships tenant templates ("World Bank programme GRM", "Corporate external GRM", "Internal HR grievance") as starting profiles

4. Worked example — one config domain (workflow excerpt, YAML)

```yaml
tenant: kisip
case_type: grievance
workflow:
  statuses:
    - {name: Received,       tag: open}
    - {name: Sorting,        tag: open}
    - {name: Investigation,  tag: in_progress}
    - {name: Escalated,      tag: in_progress}
    - {name: Referred,       tag: in_progress}
    - {name: Returned,       tag: in_progress}
    - {name: Resolved,       tag: resolved}
    - {name: Closed,         tag: closed}
    - {name: Rejected,       tag: rejected}
    - {name: In Court,       tag: on_hold}
  initial:
    default: Sorting
    rules:
      - {if: {flag: in_court}, then: In Court}
  transitions:
    - from: [Sorting, Investigation, Returned]
      to: Escalated
      roles: [grm_officer]
      effects: [{move_level: up}, {restart_sla: stage}]
    - from: [Investigation]
      to: Resolved
      roles: [grm_officer]
      requires: {fields: [resolution_summary], attachments: [signed_resolution_form]}
    - from: [Resolved]
      to: Closed
      roles: [grm_officer_national]
      guard: confirmation        # see closure policy
  closure:
    confirmation:
      required_when: {resolved_at_level_below: national}
      authority_level: national
      capture: [confirmation_notes]
    satisfaction: {enabled: true, channels: [sms, portal]}
  appeal:
    enabled: true
    window_days: 30
    routes_to: next_level
sla:
  plan: standard
  stage_durations: {Sorting: 7d, Investigation: 14d, Escalated: 14d, Resolved: 21d}
  calendar: kenya_public_holidays
```

The same schema expresses KUSP2's flow (statuses `Open/Pending`, `Under Investigation`, `Resolved`, `Referred`, `Escalated`, `Closed`; 14-day response / 30-day resolution SLA plan; satisfaction + appeal mandatory). See spec 11 for both full profiles.

03 — Domain & Data Model

All tables carry `tenant_id` plus `created_at` / `updated_at`. Soft deletion is modeled as status/flag where business-relevant; physical deletion happens only via retention jobs. Names are indicative (snake_case, PostgreSQL assumed; any relational store works).

1. Entity overview

```
erDiagram
  tenant ||--o{ jurisdiction_level : defines
  jurisdiction_level ||--o{ jurisdiction_unit : has
  tenant ||--o{ case_type : defines
  case_type ||--o{ workflow_version : has
  tenant ||--o{ category : defines
  grievance_case }o--|| case_type : of
  grievance_case }o--|| jurisdiction_unit : located_in
  grievance_case }o--o{ category : classified_as
  grievance_case ||--o{ case_event : history
  grievance_case ||--o{ thread_entry : conversation
  grievance_case ||--o{ attachment : evidence
  grievance_case ||--o{ task : work
  grievance_case ||--o{ referral : handoffs
  grievance_case ||--o| appeal : may_have
  grievance_case ||--o| satisfaction_record : feedback
  grievance_case }o--o| party : complainant
  grievance_case ||--o{ notification_log : messages
  grievance_case ||--o{ sla_clock : timers
  user_account }o--o{ role_assignment : holds
  role_assignment }o--|| role_def : of
  role_assignment }o--o| jurisdiction_unit : scoped_to
```

2. Core entities

2.1 `tenant`

`id, code, name, status, default_locale, locales[], timezone, deployment_model, created_at`

2.2 Jurisdictions

- **`jurisdiction_level`** — `id, tenant_id, rank (1=lowest), code, name_i18n, is_intake_default, is_confirmation_authority, can_be_assigned`
- **`jurisdiction_unit`** — `id, tenant_id, level_id, parent_unit_id, code, name, geo (nullable geometry), active`

The unit tree is the backbone for routing, scoping and reporting. Arbitrary depth; KISIP would load 3 levels (settlement/county/national), KUSP2 3 levels (municipality/county/national), a corporate client maybe 2.

2.3 Taxonomy

- **`case_type`** — `id, tenant_id, code, name_i18n, numbering_pattern, default_workflow_id, default_sla_plan_id, anonymity_policy, active`
- **`category`** — `id, tenant_id, parent_id, code, name_i18n, sensitivity_class_id, default_department_id, sla_plan_override_id, routing_hint, active`
- **`sensitivity_class`** — `id, tenant_id, code (standard|gbv_seah|corruption|custom...), policy jsonb` (visibility, redaction, reporting mode, designated roles — spec 07)
- **`priority_def`** — `id, tenant_id, rank, code, name_i18n, sla_plan_override_id`
- **`list_def` / `list_item`** — admin-defined controlled lists for custom fields

2.4 The case

`grievance_case` (the aggregate root; deliberately *thin* — workflow state, not workflow rules):

Column group	Columns
Identity	`id (uuid)`, `reference (unique per tenant)`, `case_type_id`, `workflow_version_id` (pinned at creation)
Classification	`categories[] (m2m)`, `priority_id`, `sensitivity_class_id` (denormalized from category, overridable upward only), `expected_outcomes[]`
Location	`jurisdiction_unit_id` (where it occurred), `current_level_id` (where it is being handled — escalation moves this), `location_text`, `geo (nullable)`
State	`status_id`, `status_since`, `previous_status_id`, `on_legal_hold bool`
People	`complainant_party_id (nullable — anonymous)`, `submitted_by_user_id (nullable — staff-assisted)`, `assignee_user_id`, `assignee_team_id`, `department_id`, `collaborators[]`
Submission	`channel`, `submitted_at`, `received_at`, `consent_record_id`, `is_anonymous`, `representative jsonb (nullable)`, `reported_elsewhere jsonb`
Content	`summary`, `description`, `custom_fields jsonb` (validated against the case type's form definition)
Resolution	`resolution_summary`, `resolved_at`, `closed_at`, `closure_reason_id`, `confirmation jsonb (by, at, level, notes)`
Flags	`is_duplicate_of (case id)`, `merged_into (case id)`, `reopen_count`

Constraints: `reference` unique per tenant; a partial unique index for duplicate suppression is **per-tenant configurable** (expression index built from the configured dedupe rule), replacing KISIP's hardcoded `(description, settlement_id, age, gender, phone)` constraint.

2.5 Parties and PII

`party` — a person or organization referenced by cases: `id, tenant_id, kind (person|organization), name_enc, national_id_enc, phone_enc, email_enc, address_enc, gender, age_band, vulnerable_group_tags[], preferred_contact_method, preferred_locale, user_account_id (nullable — registered complainant)`

- PII columns are **encrypted at the application layer** (per-tenant data key, envelope encryption; never SQL-string-interpolated keys as in KISIP).
- Searchable lookups use salted hashes (e.g. `phone_hash`) — supports "find cases by phone" without decrypting at query time.
- A party can have many cases; this gives complainant history (KUSP2 complainant directory FR-USER-01) and repeat-complainant analytics.

`consent_record` — `id, party_id (nullable for anonymous), case_id, privacy_notice_version, checkboxes jsonb, captured_at, captured_channel, captured_by`

2.6 History & conversation

- **`case_event`** (append-only audit of the case): `id, case_id, seq, kind (created|status_changed|assigned|level_moved|edited|escalated|referred|appealed|reopened|confirmed|closed|sla_breached|merged|...), actor_user_id (nullable=system/public), payload jsonb (before/after for edits), at`
- Replaces KISIP's three overlapping tables (`grievance_log`, `grievance_history`, controller-side logging) with one source of truth. "Revert edit" replays an inverse payload as a new event.
- **`thread_entry`** (communication): `id, case_id, direction (inbound|outbound|internal_note), channel, author_user_id/party_id, body_richtext, visibility (public|staff), attachments[], at`
- Internal notes are never exposed to public surfaces — enforced at the query layer, not the UI.
- **`task`**: `id, case_id, title, description, assignee, due_at, priority, status (open|done|cancelled), created_by`
- **`attachment`**: `id, case_id, thread_entry_id (nullable), task_id (nullable), kind (evidence|signed_resolution_form|acknowledgement|other — tenant-extensible), filename, mime, size, sha256, storage_key, visibility, malware_scan_status, uploaded_by, uploaded_at` — binary content in object storage, access always via authorizing endpoint with audit.

2.7 Workflow & SLA state

- **`workflow_version`**: `id, case_type_id, version, definition jsonb (statuses, transitions, policies), status (draft|active|retired), activated_at, activated_by`
- **`status_def`** (projection of definition for FK integrity): `id, workflow_version_id, code, name_i18n, tag`
- **`sla_plan`** / **`calendar`**: as configured (CD-05); calendars hold working hours + holiday dates.
- **`sla_clock`**: `id, case_id, kind (acknowledge|first_response|resolution|stage), started_at, due_at, paused_periods jsonb, satisfied_at, breached_at`
- Due dates are **computed and stored server-side** by the SLA engine; clients never supply expiry dates (KISIP let the browser set `status_expiry_date` — root cause of its broken SLA data).

- **`escalation_rule`** / **`escalation_event`**: rule config + an audit row each time a rule fires (KISIP's empty `grievance_escalation` table becomes real).

2.8 Loop-closing entities

- **`referral`**: `id, case_id, kind (internal_unit|external_institution), target_institution_id/target_unit_id, reason, sent_at, external_reference, outcome, returned_at`
- **`appeal`**: `id, case_id, round, raised_by (party|staff), reason, raised_at, routed_to_level_id, decision, decided_by, decided_at`
- **`satisfaction_record`**: `id, case_id, response (satisfied|not_satisfied|na_anonymous|no_response), channel, captured_at, comment`
- **`committee`** / **`committee_member`** / **`committee_decision`**: optional module; decisions link to `case_event`s.

2.9 Notifications

- **`notification_rule`** (config, CD-09) and **`notification_log`**: `id, case_id (nullable), event_kind, recipient_kind (party|user|role_broadcast), recipient_address_hash, channel, template_id, locale, rendered_preview (PII-redacted), status (queued|sent|delivered|failed|suppressed:reason), provider_message_id, attempts, at`
- Keeps KISIP's best feature — every send (including suppressed ones) is auditable — generalized to all channels.

2.10 Identity & access

- **`user_account`**: `id, tenant_id, username/email, phone, password_hash, mfa jsonb, sso_subject, status, locale, last_login_at`
- **`role_def`**: `id, tenant_id, code, name, permissions[] (from fixed catalogue), sensitive_classes[] (which sensitivity classes this role may see)`
- **`role_assignment`**: `id, user_id, role_id, jurisdiction_unit_id (nullable=tenant-wide), department_id (nullable), valid_from, valid_to`
- Time-bounded assignments (KISIP's role-expiry concept, kept).
- **`department`** / **`team`** / **`team_member`**: routing containers.
- **`api_key`**: scoped keys for partner integrations.

2.11 Platform & ops

- **`audit_event`** (system-wide, beyond per-case events): logins, config changes, exports, sensitive-case views, permission denials. `actor, action, object_type/id, ip, user_agent, at, detail jsonb`. Append-only, hash-chained for tamper evidence.
- **`config_version`**: registry storage (domain, version, body jsonb, status, author, note).
- **`knowledge_article`** / **`faq`** / **`canned_response`**: knowledge module.
- **`dashboard_def`** / **`dashboard_section`** / **`dashboard_widget`**: admin-built dashboard config (CD-15); widget definitions reference semantic-layer datasets, never raw tables (spec 08 §2).
- **`ai_interaction`**: audit record for every AI call (capability, provider/model+version, input hash, suggestion, confidence, accept/reject + deciding user) and chatbot transcripts linked to cases (spec 05 §7).
- **`saved_view`**: per-user/shared queue definitions (filters, columns, sort).
- **`retention_policy`** / **`retention_job_run`**: scheduled anonymization/archival with run logs.

3. Key modeling decisions (vs. KISIP)

Decision	Rationale
`case_event` single audit stream	KISIP splits history across `grievance_log`, `grievance_history` and notifications; reconstruction requires joins and guesswork
Workflow pinned per case (`workflow_version_id`)	Config changes never corrupt in-flight cases; explicit migration mapping if the tenant wants to move them
Levels/units as data, not columns	KISIP's `county_id/subcounty_id/ward_id/settlement_id` columns hardwire Kenya's geography; the unit tree supports any client
Party separated from case	Enables anonymous cases (null party), complainant history, registered accounts, and clean PII encryption boundaries
SLA clocks server-computed	KISIP trusts browser-supplied expiry dates (and has a live unit bug)
Sensitivity as a class with policy, not a boolean	KISIP's `isgbv` boolean can't express corruption-case rules or per-class reporting modes
Status semantic tags	Cross-tenant reporting ("% resolved") works even though tenants name statuses differently

4. Indexing & volume notes

- Hot paths: queue queries (`tenant_id, status_tag, current_level_id, assignee`), reference lookup (`tenant_id, reference`), party lookup by `phone_hash` .
- `case_event` , `notification_log` , `audit_event` are append-only → BRIN/partitioning by month at scale.
- Sizing baseline per KUSP2: hundreds of staff users, tens of thousands of cases/year per tenant — modest; design for 100× headroom (NFR-02).

04 — Workflow Engine: Lifecycle, SLA, Escalation, Appeals, Closure

The workflow engine executes each tenant's configured case lifecycle **server-side**. The UI renders available actions by asking the engine; it never computes permissions, transitions, levels or due dates itself.

1. Reference lifecycle (default template)

Shipped as the platform default (tenants override freely). It encodes the GRM doctrine common to KISIP, KUSP2, NAVCDP and the World Bank guidance:

```
stateDiagram-v2
    [*] --> Received: any channel
    Received --> Screening: auto/staff triage
    Screening --> Rejected: ineligible / out of scope (with referral guidance)
    Screening --> InProgress: eligible, assigned
    InProgress --> Escalated: SLA breach / severity / manual
    Escalated --> InProgress: handled at higher level
    InProgress --> Returned: sent back down a level
    Returned --> InProgress
    InProgress --> ReferredOut: external institution
    ReferredOut --> InProgress: outcome returned
    InProgress --> Resolved: resolution recorded (+ evidence)
    Resolved --> Closed: confirmation + satisfaction + appeal window lapsed
    Resolved --> Appealed: complainant not satisfied (within window)
    Appealed --> InProgress: re-opened at higher level
    Closed --> Reopened: within reopen policy
    Reopened --> InProgress
```

Key doctrine points enforced by the default policies:

1. **Explicit screening step** with an ineligibility outcome that records *why* and *where the complainant was redirected* (gap in KISIP; required by NAVCDP/Cassava/KUSP2).
2. **Resolution ≠ closure.** Closure requires (configurably): resolution summary + evidence, complainant notified, satisfaction captured (or N/A-anonymous), appeal window lapsed or waived, and higher-level confirmation where configured (KISIP's national-confirmation pattern, generalized).
3. **Escalation moves the handling level up the jurisdiction tree; Return moves it down.** Direction comes from the configured hierarchy (CD-02), never from hardcoded names.

2. The action API (atomicity)

Every staff operation is a single **case action** request:

```
POST /api/v1/cases/{id}/actions
{ "action": "transition", "to_status": "Resolved",
  "note": "...", "fields": {...}, "attachments": [...],
  "assignee": ..., "idempotency_key": "..." }
```

The engine, in **one database transaction**:

1. Loads the case + its pinned `workflow_version`.
2. Evaluates **guards** (see §3). Any failure → typed error, nothing persisted.
3. Applies the transition: status, level moves, assignee changes, resolution fields.
4. Writes the `case_event` (+ thread entry if a note/reply was included, + attachments).
5. Updates SLA clocks (stop/satisfy/restart per transition config).
6. Enqueues notifications (outbox pattern — delivery is async, enqueueing is transactional).

No client-orchestrated multi-call sequences and no client-side rollback (KISIP's pattern). `idempotency_key` makes retries safe.

3. Guards (transition preconditions)

Configured per transition; evaluated server-side:

Guard	Examples
Role	only roles listed may execute
Level	actor's jurisdiction scope must cover the case's `current_level` /unit (national acts anywhere; county on its subtree; etc.)
Required fields	`resolution_summary` for Resolve; `reason` for Reject/Return
Required attachments	attachment of kind `signed_resolution_form` for Resolve (KISIP practice, kept as config)
Required note	free-text justification
Case state	no open blocking tasks; not on legal hold; appeal window state
Approval	transition requires second-person approval when priority ≥ X or sensitivity ∈ Y (KUSP2 closure checklist / supervisory approval)
Committee decision	a committee decision record must be attached (optional module)

4. SLA engine

4.1 Clocks

Per case, up to four clock kinds (per SLA plan):

Clock	Starts	Satisfied by
`acknowledge`	submission	acknowledgement notification delivered (or manual ack)
`first_response`	submission	first outbound `thread_entry`
`resolution`	submission (or eligibility confirmation — configurable)	entering a `resolved` -tagged status
`stage`	each status entry (if stage durations configured)	leaving the status

4.2 Computation rules

- Due dates = start + target, computed over the bound **calendar** (working hours/holidays) when the plan says working time; calendar time otherwise.
- Stored on `sla_clock`; **never** accepted from clients.
- Pauses: configurable statuses (e.g. `Awaiting Complainant Information`, `In Court`) pause specified clocks; pause periods are recorded.
- Changing priority/category mid-case re-resolves the SLA plan (configurable: recompute vs keep).

4.3 Breach handling

A scheduler (fixed cadence, e.g. every 15 min — not KISIP's random weekly cron) evaluates clocks:

- **At-risk** (configurable lead time, e.g. T-2 days): reminder notification to assignee; case flagged in queues.
- **Breach**: `sla_breached` case event; notification per rules; **escalation rules** evaluated.
- All evaluations are idempotent and logged.

5. Escalation rules

Declarative, per tenant (CD-05):

```
- name: overdue-auto-escalate
  trigger: {clock: stage, state: breached}
  condition: {status_tag: in_progress, level_below: top}
  action:
    - move_level: up
    - set_status: Escalated
    - notify: {role: grm_officer, scope: new_level_unit}
- name: emergency-priority
  trigger: {on: case_created}
  condition: {priority: Emergency}
  action: [{notify: {role: supervisor, scope: unit}}, {set_sla_plan: emergency}]
- name: dissatisfaction-appeal
  trigger: {on: satisfaction_recorded}
  condition: {response: not_satisfied}
  action: [{open_appeal: {route: next_level}}]
```

Triggers: clock states, case events, manual invocation. Actions: move level, set status, reassign, change priority/SLA plan, notify, open appeal, create task. Every firing writes an `escalation_event`.

Manual escalation remains a normal transition with guards; complainant-initiated escalation/appeal comes through the public surface (§7).

6. Closure pipeline (configurable stages)

```
Resolved → [confirmation] → [satisfaction capture] → [appeal window] → Closed
```

Stage	Config	Behavior
Confirmation	`required_when` (e.g. resolved below level X), `authority_level`, captured fields	Generalizes KISIP's national confirmation. Confirmation is itself a guarded action with its own audit fields
Satisfaction	enabled, channels (SMS reply, portal link, staff phone capture), reminder schedule, timeout behavior	Outcome stored on `satisfaction_record`; `not_satisfied` can trigger appeal rule; anonymous → `na_anonymous`
Appeal window	days, who may appeal, max rounds, routing	While open, case sits in `Resolved`; lapse → auto-transition to `Closed` (system actor)
Closure	checklist fields, closure reason code, supervisory approval thresholds	Closing writes `closed_at`; retention clock starts

Reopen policy: who (complainant within N days / staff with permission), resulting status, increment `reopen_count`, full audit.

7. Public/complainant-initiated actions

Via the public portal/USSD/hotline (verified by reference + verifier; spec 05):

Action	Behavior
Add information / reply	Creates inbound `thread_entry`; may un-pause `awaiting information` status per config
Express dissatisfaction / appeal	Within window → `appeal` record + routing; outside window → recorded, staff notified
Self-escalate	If tenant enables it (KISIP feature): guarded equivalent of escalation, rate-limited, audited with `actor=complainant`
Withdraw	Transition to closure with reason `withdrawn` (confirmation prompt)

8. Engine guarantees

1. **Determinism**: same case + same action + same config version → same result.
2. **Total auditability**: every state change has exactly one `case_event` with actor and before/after.
3. **No orphan states**: validation (spec 02 §3) guarantees a path to closure from every reachable status.
4. **Version isolation**: in-flight cases run on their pinned workflow version until explicitly migrated.
5. **Clock integrity**: all SLA timestamps system-computed; UI displays only.
6. **Concurrency**: optimistic locking on the case row (`version` column); conflicting concurrent actions fail cleanly with retry guidance.

05 — Intake & Channels

All channels normalize into one **standardized intake dataset** and one unified case register (KUSP2 §4.1). Channels are per-tenant modules (CD-08); the baseline every tenant gets is **web portal + staff-assisted intake**; telecom channels activate when the client's arrangements allow.

1. Channel matrix

Channel	Mode	Tenant config	Minimum capture
Web portal	Self-service	Theme, locales, form per case type, CAPTCHA, knowledge base on/off, registered accounts on/off	Full configured form
Assisted intake	Staff on behalf (walk-in, phone, letter, community meeting, complaint box)	Source-channel list, intake scripts (read-back confirmation), draft save	Full form, `submitted_by_user_id` recorded, source channel mandatory
USSD	Self-service, feature phones	Gateway provider, shortcode, menu tree per locale	Location + summary + category (configurable minimum); case flagged `requires_completion` for staff follow-up
SMS	Notifications always; optional structured intake	Provider profile, sender ID, opt-in rules	If intake enabled: free-text → `requires_completion` triage queue
Hotline / IVR	Operator-assisted	Telephony metadata fields (call id, operator), scripts, callback policy	As assisted intake
Email (inbound)	Semi-structured	Monitored mailboxes, parsing rules, auto-ack	Subject/body → summary/description; `requires_completion`
Chatbot (opt-in, §7)	Conversational self-service (web widget, WhatsApp)	AI provider profile, persona/tone, languages, scope of questions, escalation-to-human rules	Guided slot-filling to the same channel minimum as USSD; transcript attached; `requires_completion` if incomplete
Partner API	System-to-system	API keys, field mapping, allowed case types	Full dataset per API contract

Chatbot intake is **default-off platform-wide** and activates only by explicit tenant opt-in (KUSP2 prohibits it via FR-PUB-15 — its tenant profile keeps the flag off; other tenants may enable it).

2. Intake pipeline (all channels)

```
capture → validate (form def) → consent (if PII) → dedupe check → create case
  → reference number → initial status & level (workflow rules) → routing
  → acknowledgement (notification engine) → [requires_completion triage if minimal channel]
```

Step	Specification
Validate	Against the case type's form definition (CD-06): required/conditional fields, types, list values. Channel minimums override (USSD/hotline/email)
Consent	If any PII captured: consent checkboxes per config, stored on `consent_record` with privacy-notice version. Anonymous path skips PII and consent
Dedupe	Configured rule (e.g. same phone + similar description within N days). Outcomes per config: warn submitter, auto-link as related, or queue for staff merge decision. Never a silent DB error (KISIP's unique-constraint approach)
Reference	Per numbering pattern (CD-07). Sequence allocation is concurrency-safe (DB sequence per scope), not max()+1 string parsing (KISIP)
Initial status/level	From workflow initial rules: default status; flag/category overrides (e.g. sensitive class starts at designated level — generalizes KISIP's GBV→national)
Routing	Category → department/team defaults; routing rules (filters) may set assignee, priority, SLA plan, add internal note
Acknowledgement	Immediate, via the submitter's channel + configured extras: reference number, what happens next, tracking instructions. Satisfies the `acknowledge` SLA clock

3. Standardized intake dataset (base form)

Superset of KUSP2 Appendix A and the KISIP form; every field is per-tenant configurable (enabled/required/label/help):

Section A — Complainant (all optional to permit anonymity, unless tenant requires) - Name; phone; email; postal/physical address; preferred contact method; preferred language - Gender; age band; vulnerable-group tags (tenant-defined list) - National ID (encrypted; tenants

enable only if justified) - Representative: name, relationship/organization, contact, consent-of-affected-person - **Consent to personal data use** (required whenever PII present)

Section B — Grievance - Submission channel (system-set); date of occurrence; date of submission (system) - Jurisdiction unit (cascading selector over the tenant hierarchy); location/site free text; optional geo-point - Category (multi-select per taxonomy); case type; sensitivity auto-derived - Summary; description; evidence attachments - Reported elsewhere? (where/reference) - Tenant custom fields (CD-06)

Section C — Expected outcome (multi-select + free text)

Section D — Official use (assisted intake / triage) - Received by, received at, source channel detail; initial assignment; internal notes

4. Public portal — functional spec

Feature	Spec
Home	Process overview (configurable stages content), free-of-charge + non-retaliation + confidentiality assurances (presence enforced), CTAs: submit / track / knowledge base
Submit	Multi-step configured form; anonymous toggle (if allowed); attachment upload; CAPTCHA; confirmation page with reference + verifier guidance; print/download acknowledgement
Track status	Reference + verifier (phone/email used at submission, or PIN issued at intake for anonymous cases). Shows: status (public label), stage timeline, public thread entries, documents marked public. PII minimized; sensitive cases show stripped view per sensitivity policy. Generic errors prevent enumeration; rate-limited
Complainant actions	Reply/add info; appeal/dissatisfaction (when window open); withdraw — per workflow config (spec 04 §7)
Registered accounts (optional)	Email/phone OTP signup; case history; notification preferences
Knowledge base (optional)	Categories, search, featured articles, multi-language
Accessibility & reach	WCAG 2.1 AA; low-bandwidth pages (no heavy assets on critical paths); responsive; all enabled locales

Anti-abuse: per-IP and per-identity rate limits on submission and lookup; attachment scanning (if enabled); honeypot fields; audit of rejected attempts.

5. Staff console — intake-related spec

- **Assisted intake form** = full form + Section D; party lookup-or-create (by phone/email hash) to link complainant history; read-back confirmation step per script config; draft save.
- **Completion queue**: cases flagged `requires_completion` (USSD/SMS/email) listed for enrichment; completing them is a guarded action.
- **Merge/link**: duplicate handling UI — link related cases or merge (merged case closes with reason `duplicate/merged`, thread preserved, audit event both sides).

6. Channel adapter contract

Each channel adapter implements:

```
normalize(raw) -> IntakeSubmission     # standardized dataset subset + channel metadata
verify(raw)    -> abuse/validity signals
ack(case)      -> channel-appropriate acknowledgement payload
```

Adapters are stateless against the intake pipeline; new channels (e.g. WhatsApp later) are added by implementing the contract + a config block, with no core changes.

7. Chatbot & AI assistance (opt-in modules, CD-16)

Both are feature-flagged off by default and configured per tenant. Precedent: the KISIP codebase already runs an AI document service (separate `document_ai` Postgres database, chunked document embeddings, OpenAI/xAI keys) — the generic platform formalizes that pattern behind governance rules.

7.1 Chatbot intake (conversational channel)

A chatbot is **just another channel adapter** feeding the standard intake pipeline — it never has its own case-creation path.

Aspect	Specification
Surfaces	Web portal widget; WhatsApp/Telegram via gateway adapter; same engine behind both
Conversation design	Guided **slot-filling** over the case type's form definition (CD-06): the bot asks for the configured channel-minimum fields (like USSD), confirms a read-back summary, then submits. Free-form storytelling is allowed — the AI extracts candidate field values, but every extracted value is **confirmed with the complainant before submission**
Scope control	Tenant-configured allowed intents: file a case, check status (reference + verifier, same rules as portal tracking), answer FAQs from the knowledge base (RAG over published articles only). Anything else → polite refusal + handoff
Human handoff	Always available ("talk to a person") → creates an assisted-intake callback request or transfers to hotline; mandatory automatic handoff when sensitivity signals fire (§7.3)
Transparency	The bot identifies itself as automated at conversation start (configurable text, presence enforced — same pattern as the free-of-charge statement); transcript stored on the case as channel metadata
Anonymity & consent	Anonymous path supported per tenant policy; consent captured conversationally before any PII is taken, recorded on `consent_record` like any channel

7.2 AI assistance for staff & triage

Each capability is an independent flag (tenant may enable some and not others):

Capability	What it does	Guardrail
Auto-categorization	Suggests category, case type, priority from the description	Suggestion only — pre-fills triage fields, staff confirm; confidence shown
Sensitivity detection	Flags probable GBV/SEA-SH, corruption, threats-of-harm content at intake	High-recall trigger: a positive flag immediately applies the sensitivity class's *restrictions* (visibility, redaction) pending human confirmation; never auto-dismisses
Semantic duplicate detection	Embedding similarity against recent cases, augmenting the rule-based dedupe (spec 05 §2)	Same outcomes as configured dedupe rule: warn / link / merge queue
Summarization	Case summary, timeline digest, thread recap for handover/escalation	Labeled as AI-generated; regenerable; never replaces the original text
Translation & language detection	Detects submission language; provides working translations for staff	Original always preserved; outbound complainant messages use human-approved templates, not machine translation, unless tenant opts in
Draft responses	Suggests replies/resolution summaries from canned responses + case context	Staff edit/approve before send; drafts logged
KB answer assist (RAG)	Answers public FAQ questions from published knowledge-base articles with citations	Published, non-sensitive content only; "I don't know" fallback

7.3 AI governance (platform rules, non-configurable)

1. **Human in the loop for every case decision.** AI never changes status, assigns, closes, rejects, or sends complainant-facing free text on its own. Its outputs are suggestions that a permissioned human accepts — acceptance is the audited action.
2. **Sensitive-case fail-safe.** Sensitivity signals escalate restrictions immediately but only a designated human can clear them. Sensitive-case content is excluded from AI processing unless the tenant's sensitivity policy explicitly allows it (and then only via the approved provider).
3. **PII minimization.** PII fields are stripped/pseudonymized before prompts leave the platform; provider profile declares data residency and a no-training contractual flag; on-prem/local model deployment supported for gov-hosted tenants.
4. **Full audit.** Every AI call logged: capability, model/provider+version, input hash, suggestion, confidence, accept/reject decision and deciding user. Suggestions are reproducible evidence in appeals.
5. **Evaluation & drift.** Per-capability quality dashboards (acceptance rate, override rate, false-positive sensitivity flags); tenant can disable any capability instantly (kill switch, audited like notification switches).

CD-16 config block (summary)

Provider profiles (OpenAI/Azure/xAI/local; keys in vault), per-capability enable flags + model selection, chatbot persona/locales/intents/handoff rules, PII redaction policy, sensitivity-processing policy, confidence thresholds, RAG corpus selection.

06 — Notification Engine

Declarative, per-tenant notification rules replace inline notification code. Every message — sent, failed, or suppressed — is logged.

1. Model

```
event → rule(s) → recipients → template (locale, channel) → delivery → log
```

1.1 Events (emitted by the engines; fixed catalogue, extensible)

`case.created`, `case.acknowledged`, `case.status_changed`, `case.assigned`, `case.level_moved` (escalated/returned), `case.referred_out`, `case.resolved`, `case.confirmation_requested`, `case.confirmed`, `case.closed`, `case.reopened`, `appeal.opened`, `appeal.decided`, `satisfaction.requested`, `sla.at_risk`, `sla.breached`, `thread.reply_external`, `thread.reply_inbound`, `task.assigned`, `task.overdue`, `committee.decision_recorded`, plus admin events (`user.invited`, `config.changed`, …).

1.2 Rules (CD-09)

```
- on: case.created
  to: [{party: complainant}, {role: grm_officer, scope: case_unit_and_above}]
  channels: {party: [sms, email], staff: [email, in_app]}
  template: case-registered
  condition: {sensitivity: standard}      # sensitive classes use redacted variants
- on: sla.at_risk
  to: [{user: assignee}]
  channels: [email, in_app]
  template: case-at-risk
- on: case.status_changed
  to: [{party: complainant}]
  channels: [sms]
  template: status-update
  condition: {not_status: [Referred]}     # KISIP rule "don't tell complainant about internal referral", now config
```

Recipient selectors: `party:complainant|representative`, `user:assignee|case_creator`, `role:<role> scope:<case_unit|unit_and_above|level|tenant>`, `team:<team>`, explicit address (admin alerts). Selector resolution uses role assignments + jurisdiction scoping — replacing KISIP's hardcoded `roleid: 4` queries.

Conditions: category, sensitivity class, priority, level, channel of submission, status (from/to), anonymous flag.

1.3 Templates

- Per template: per **locale** and per **channel** variant (SMS short form, email rich form, in-app).
- Variables: case reference, status label (localized), jurisdiction names, dates, tracking link, actor display name, tenant branding tokens. Unknown variables fail validation at config time.
- **Privacy modes**: each template is marked `standard` or `privacy_safe`; sensitivity classes force `privacy_safe` variants (reference + generic status only — no names, no narrative; KUSP2 FR-NOTIF-02/FR-SENS).
- Tracking links: signed, time-limited URLs to the public status page (no guessable numeric IDs — KISIP lesson).

2. Delivery

Aspect	Specification
Outbox pattern	Notifications enqueue inside the case-action transaction; a worker delivers asynchronously with retries (exponential backoff, max attempts configurable)
Channel providers	Pluggable adapters: SMS (e.g. Advanta, Africa's Talking, Twilio), email (SMTP per tenant identity, SPF/DKIM guidance), in-app, USSD push where supported. Provider profiles are tenant config; one interface
Kill switches	Per tenant × channel × module (e.g. disable SMS for grievance events at county level). Suppressed messages are still logged with `suppressed:<by whom/why>` (KISIP pattern, kept)
Quiet hours	Optional per tenant (queue until morning) except emergency-priority events
Delivery status	Provider callbacks update `notification_log.status` (sent → delivered/failed); failures surface in an admin queue (KUSP2 INT-08/09)

Aspect	Specification
Throttling/dedup	Same event + recipient + template within a window collapses to one message; per-recipient daily caps configurable

3. Logging & audit

`notification_log` (spec 03 §2.9) records every attempt: event, recipient (hashed address), channel, template + locale, redacted preview, status timeline, provider message id, case linkage. Admin console: searchable log, failure dashboard, resend action (guarded).

4. Defaults shipped with the platform

A default rule pack implementing the doctrine baseline: complainant ack on creation (with tracking link), status-change notices (excluding internal-only transitions), officer notices on assignment/escalation/return/referral, at-risk and breach reminders, satisfaction request on resolution, closure notice, appeal notices. Tenants edit from there.

07 — Security, Access Control & Sensitive Cases

1. Authentication

Item	Specification
Staff auth	Username/email + password (tenant policy: length, rotation, lockout) ; **MFA configurable per role, mandatory for privileged roles** (admins, sensitive-case roles)
SSO	OIDC and SAML 2.0 per tenant; directory-group → role mapping; secure local fallback accounts (KUSP2 §16.5)
Complainant auth	Optional registered accounts (email/phone OTP); guest tracking via reference + verifier or signed magic links
Sessions	Short-lived access tokens + refresh; absolute and idle timeouts per tenant; concurrent-session policy
API	Scoped API keys / OAuth client credentials; IP allowlists per key
Perimeter	Optional IP ACLs for staff/admin consoles (KUSP2 NFR-07); CAPTCHA + rate limits on public surface

Hard rule: every endpoint requires authentication except the designated public surface (submit, track, knowledge base, satisfaction/appeal links), each of which is rate-limited, abuse-monitored and audited. (KISIP exposes `/log`, `/upsert`, `/upload`, `/self/escalate` unauthenticated — explicitly forbidden here.)

2. Authorization model

2.1 Fixed permission catalogue (platform code)

Permissions are fine-grained verbs the platform understands; tenants compose them into roles (CD-10). Families:

```
case:read, case:create_assisted, case:transition, case:assign, case:edit_fields,
case:read_pii, case:export, case:merge, case:delete_soft, case:restore,
case:confirm_resolution, case:override_sla, case:legal_hold,
thread:reply_external, thread:note_internal, attachment:upload, attachment:read_protected,
task:*, appeal:decide, referral:manage, committee:record_decision,
report:view_operational, report:view_aggregate, report:export,
admin:config_*, admin:users, admin:roles, admin:notifications, admin:audit_read,
sensitive:<class>:read, sensitive:<class>:handle      # generated per sensitivity class
```

2.2 Three-axis scoping

Every grant = **role (permission set) × jurisdiction scope × sensitivity clearance**:

1. **Permissions** — what verbs.
2. **Jurisdiction scope** — which subtree of the unit tree (assignment to a unit covers that unit and below; tenant-wide = root). Replaces KISIP's county_id/settlement_id role columns.
3. **Sensitivity clearance** — which sensitivity classes the role may see/handle.

Enforcement is at the **data-access layer** (query filters + row-level checks), not UI: list endpoints silently scope; detail/action endpoints return 403/404 consistently (404 for sensitive cases the user can't know exist).

2.3 Record-level rules (beyond scope)

- Assignee/collaborators always see their cases (within sensitivity clearance).
- Department/team access lists (KUSP2 department access).
- Contractors/service providers: restricted role template — assigned cases only, no PII fields, no export.

3. Sensitive-case policies (sensitivity classes)

Each class (CD-03) defines a policy bundle; `gbv_seah` ships as a strict default:

Policy knob	`standard`	`gbv_seah` **default**
Visibility	jurisdiction scoping only	**only roles with explicit clearance**; others: case invisible (404), excluded from search/keyword/queues
Intake routing	normal initial level	starts at **designated level** (e.g. national) and routes to designated focal roles

Policy knob	`standard`	`gbv_seah` **default**
PII display	per `case:read_pii`	masked for everyone except clearance + `read_pii`; full view events audited individually
Free text	normal	**minimal-capture form variant** (structured referral fields, restrained narrative)
Notifications	standard templates	**privacy-safe templates forced** (no names/narrative in SMS/email)
Reporting	full disaggregation	**aggregate-only**, minimum cell size (e.g. suppress counts < 5)
Referrals	optional	structured referral to service-provider directory; survivor consent flags; follow-up consent
Break-glass	n/a	optional: emergency access with mandatory justification, auto-alert to DPO role, prominent audit
Export	per role	blocked except aggregate

Other classes (e.g. `corruption`) configure their own bundle (e.g. restricted to integrity unit, external referral to EACC tracked).

4. PII protection

Item	Specification
Field-level encryption	PII columns encrypted at application layer (AES-GCM), per-tenant data keys, envelope encryption with KMS/HSM where available; key rotation procedure. No SQL-interpolated keys (KISIP defect)
Search on PII	Salted keyed hashes (HMAC) for exact lookup (phone/email/national-id); no plaintext indexes
At rest / in transit	Disk/db encryption; TLS 1.2+ everywhere; HSTS; secure cookies
Redaction	Role-driven field masking applied server-side in serializers (single choke point)
Consent & DSAR	Consent records versioned (spec 05); data-subject access/erasure workflows: locate by party, export, anonymize (subject to legal hold + retention) — Kenya DPA 2019 alignment
Retention	Per case type/sensitivity policies (CD-13): anonymize (strip party + PII fields, keep stats) or archive or delete; runs logged; legal hold blocks
Backups	Encrypted; restore tested before go-live and periodically (KUSP2 DEL-13)

5. Audit

Item	Specification
Coverage	All case events; all config changes (diffs); logins/failures; permission denials; exports (who, what fields, row count); **every view of a sensitive case**; notification sends; retention runs
Integrity	Append-only; hash-chained per tenant; no UPDATE/DELETE grants on audit tables
Access	`admin:audit_read` role; filterable UI + export (REP-04/NFR-19); auditor read-only role template
Retention	Online ≥ 90 days minimum (KUSP2 24.4.5), archive per policy

6. Application & operational security

- OWASP ASVS L2 target; input validation server-side; output encoding; CSRF protection on consoles; security headers (CSP, X-Frame-Options per embedding config).
- File uploads: type/size enforcement, content sniffing, optional malware scanning (FR-ATT-03), storage outside web root, signed download URLs through an authorizing endpoint.
- Secrets in a vault/parameter store; no secrets in code or client bundles. (KISIP ships DB passwords and API keys in committed `.env` files — forbidden.)
- Dependency and image scanning in CI; vulnerability management SLAs; pen test before go-live where contracted (DEL-10).
- Environment isolation: dev/test/staging/prod; no production PII in non-prod without authorization (KUSP2 24.4.4); seeded synthetic data for testing.
- Incident response runbook; security event alerting (auth anomalies, mass export, break-glass use).

08 — Reporting, Dashboards & KPIs

Reporting works across tenants because metrics are computed on **semantic status tags** and the **jurisdiction tree**, not tenant-specific names.

1. Standard KPI set (platform-computed)

Doctrine KPIs (World Bank presentation + KUSP2 REP-07/08), available to every tenant out of the box:

KPI	Definition
Cases registered	Count by period, channel, category, unit, priority, sensitivity (aggregate-only for sensitive)
Acknowledgement time	submitted → acknowledged (clock data); median/p90; % within target
First-response time	submitted → first outbound reply; % within target
Resolution time	submitted → `resolved` tag; % within target (e.g. ≤30 days)
% resolved / closed	by period and dimension
% resolved within SLA	per clock targets
Overdue / at-risk counts	live, by unit and assignee
Escalation rate	% of cases escalated ≥1 level; auto vs manual
Referral-out rate	by institution
Appeal rate / reopen rate	per closure cohort
Complainant satisfaction	satisfied / not satisfied / N-A-anonymous / no-response distribution
Anonymous share	% anonymous submissions
Repeat complainants	parties with >N cases (non-sensitive only)
Workload	open cases per assignee/team/unit; assignment latency
GRM access proxy	submissions per 1,000 target population per unit (population figures = tenant reference data, optional)

Tenants set **targets** per KPI (CD-13); dashboards show actual vs target.

2. Configurable dashboards (CD-15)

Dashboards are **tenant-built configuration**, not code. The design generalizes the production `plus-admin` dynamic-dashboard system (`dashboard` → `dashboard_section` → `dashboard_card` / `dashboard_section_chart` rows, resolved at runtime against generic summary endpoints and rendered by a chart-type catalogue) and fixes its weaknesses.

2.1 Structure

```
dashboard (title, icon, audience, is_main, is_public, layout)
  └── section (title, icon, color, order)
        └── widget (kpi card | chart | table | map)
```

Each **widget** is a declarative definition:

Field	Meaning	plus-admin equivalent
`dataset`	A named dataset from the **semantic layer** (§2.3), e.g. `cases`, `case_events`, `sla_clocks`, `satisfaction`	`card_model` (raw model name)
`measure` + `aggregation`	field + `count` / `count_distinct` / `sum` / `avg` / `min` / `max` / `pct`	`card_model_field` + `aggregation`, `unique`, `computation`
`group_by[]`	dimensions: category, status tag, unit (any hierarchy level), channel, priority, time bucket	`groupField`, `categorized`
`time_dimension` + `bucket`	for trend charts (`submitted_at`, `resolved_at`, …; day/week/month/quarter/year)	`time_field`
`metrics[]`	multiple measures for multi-series charts	`metric_fields`
`filters[]`	field/op/value conditions, combinable	`filters` JSONB + `filter_field/value/function/option`

Field	Meaning	plus-admin equivalent
`chart_kind`	from renderer catalogue: `kpi_card, bar, stacked_bar, stacked_bar_100, line, multi_line, area, pie, donut, treemap, map, table, pyramid`	integer `type` codes
`target / thresholds`	target value + warning bands, conditional coloring	(new)
`drill_down`	optional link to a saved queue view or child dashboard	dashboard card → list navigation
`caption / source_note`	localized footer text	hardcoded "National Slum Database" subtitle (now config)

Dashboard-level config: `audience` (roles and/or jurisdiction levels), `is_main` (the post-login landing dashboard — at most one per audience), `is_public` (renders on the transparency page with extra constraints, §2.4), global filter bar (which dimensions viewers may filter: period, unit, category, phase).

2.2 Admin builder

Admin console screens (CRUD per config-governance rules in spec 02 §3): dashboard list, section editor, widget editor with **live preview** against real (scoped) data, drag-and-drop ordering/layout, duplicate-widget, and dashboard duplication across tenants via the config bundle. Widget definitions validate at save time: dataset exists, fields exist in the dataset, group-by dims are exposed by the dataset, chart kind supports the shape (e.g. `multi_line` requires `time_dimension` + `metrics[]`).

2.3 The semantic layer (governance — the key fix over plus-admin)

plus-admin's widgets post raw model/field names to generic summary endpoints (`/summary/byfield`, `/summary/group/multiple`), which will aggregate **any table in the schema** for any authenticated user, with location scoping left to the client. The generic platform instead exposes only **curated datasets**:

- A dataset = a platform-defined (versioned) view over the domain model: allowed measures, allowed dimensions, joins pre-resolved (e.g. `cases` exposes unit names at every hierarchy level), and **mandatory row-level security**: viewer's jurisdiction subtree + sensitivity clearance applied server-side on every query.
- Sensitive-class data appears only in `cases_sensitive_aggregate` (pre-aggregated, minimum cell size enforced) regardless of widget configuration.
- PII fields are never exposed as dimensions or measures.
- Query cost guards: max group-by cardinality, max time range, result row caps; per-widget result caching with explicit cache keys and TTL (keeps plus-admin's Redis-cache pattern, but server-managed).

2.4 Default dashboard pack & special dashboards

Shipped as config (tenant-editable copies of the structures below):

Dashboard	Audience	Content
Operational	handlers/supervisors	My/team queues, overdue, at-risk, today's intake, aging buckets
Management	unit & programme managers	KPI tiles vs targets, trends, drill-down by unit subtree, category heatmap, escalation/appeal funnels
Map view (optional)	management	Case density / resolution performance on the unit tree's geo data
Sensitive aggregate	safeguards roles	Aggregate-only widgets per sensitivity policy
Admin/ops	administrators	Notification failures, channel health, config changes, audit highlights, retention runs (fixed platform widgets)
Public transparency (`is_public`)	public	Tenant-approved aggregate widgets only; anonymous-safe datasets; cached; no drill-down; no global filters beyond period/unit

All dashboards respect jurisdiction scope and sensitivity clearance; drill-down stops where permission stops.

3. Reports & exports

- **Queue/list exports**: CSV/XLSX of any saved view, columns per export field policy (PII excluded by default; `case:export` + field policy gates).
- **Case file export**: single-case PDF dossier (details, timeline, thread, attachments list, signatures block) for audits/committees — replaces KISIP's ad-hoc PDF endpoints with one templated renderer (tenant-branded).
- **Scheduled reports**: definition (filters, dimensions, format, recipients, cadence); delivered via email with signed links; runs logged.
- **Standard regulator pack**: pre-built monthly/quarterly aggregate report templates (World Bank/ESS10-style), tenant-editable.

- **Data access for BI**: read-only replicated schema or warehouse export (CSV/Parquet), PII stripped/pseudonymized; documented dictionary (DEL-03).

4. Implementation notes

- Metrics computed from `case_event` + `sla_clock` streams (event-sourced facts) — no reliance on mutable case columns; nightly materialized aggregates + live counters for queue badges.
- The semantic-layer datasets (§2.3) read from these materialized aggregates where possible, falling back to live queries only for small scoped slices — so admin-built widgets cannot trigger unbounded scans.
- Dimension model: time × unit (tree rollup) × category × channel × priority × status tag × sensitivity (suppressed below cell minimum).
- Performance: dashboards within NFR targets (<3s) at 100k+ cases via pre-aggregation.

09 — API & Integrations

1. API principles

- Single documented **REST API** (OpenAPI spec published per deployment) — the consoles are themselves API clients (no privileged backdoors).
- JSON; cursor pagination; consistent error envelope with typed error codes; idempotency keys on mutations; optimistic-concurrency via `If-Match` /version.
- Versioned (`/api/v1`); additive changes preferred; deprecation policy documented.
- All endpoints tenant-scoped by auth context; rate limits per principal.

2. Surface overview

2.1 Public (unauthenticated, rate-limited, abuse-protected)

Endpoint	Purpose
`POST /public/cases`	Submit (web form / channel adapters); CAPTCHA token; returns reference + verifier guidance
`GET /public/cases/track`	Reference + verifier (or signed token) → public status view
`POST /public/cases/{ref}/reply`	Complainant adds info (verified)
`POST /public/cases/{ref}/appeal`	Appeal/dissatisfaction (verified, window-checked)
`POST /public/satisfaction/{token}`	Satisfaction response via signed link
`GET /public/kb/*`	Knowledge base
`GET /public/stats`	Transparency aggregates (if enabled)

2.2 Staff (authenticated)

Group	Endpoints (representative)
Cases	`GET /cases` (saved-view query language), `GET /cases/{id}` , `POST /cases` (assisted), `POST /cases/{id}/actions` (the single action endpoint — spec 04 §2), `GET /cases/{id}/events` , `GET /cases/{id}/available-actions` (drives UI)
Thread	`GET/POST /cases/{id}/thread` (reply vs internal note by permission)
Tasks, attachments, referrals, appeals	CRUD per permission; attachment upload via signed URLs
Parties	`GET /parties?phone_hash=…` , history
Reports	`GET /reports/kpis` , `GET /dashboards/...` , `POST /exports` (async job + signed download)

2.3 Admin

Config registry CRUD per domain (`/admin/config/{domain}`), versions, validate, dry-run, activate, export/import bundle; users/roles; notification rules/templates + test-send; audit search.

2.4 Webhooks (outbound)

Tenant-configurable subscriptions to the event catalogue (spec 06 §1.1): signed payloads (HMAC), retries with backoff, dead-letter queue visible in admin, PII-minimized payloads by default. Use cases: ministry M&E systems, national data warehouses, partner case handoff.

2.5 Inbound integrations

Integration	Contract
SMS gateway	Provider adapters (send + delivery callbacks + inbound webhook)
USSD gateway	Session webhook (menu state machine driven by channel config)
Email	SMTP out per tenant identity; IMAP/webhook in for monitored mailboxes
Telephony/call center	Assisted-intake API with call metadata fields
SSO	OIDC/SAML per tenant

Integration	Contract
Partner systems	Scoped API keys; field-mapped case creation; external-reference linking on referrals

3. Embedding & portability

- Public portal embeddable via allowlisted iframes or linked directly from programme websites (KUSP2 INT-05/16.1).
- **Full export** (NFR-17): tenant data (all entities, open formats) + configuration bundle + attachments manifest, runnable by tenant admins; documented data dictionary.
- Migration tooling: CSV importers for jurisdiction units, users, legacy cases (KISIP-style bulk import becomes an authenticated, validated, logged admin job — not an open `/upsert` endpoint).

10 — Requirements Catalogue (Traceable)

Priorities: **M**ust (MVP), **S**hould (fast-follow), **C**ould (differentiator). Source mapping: K2 = KUSP2 requirement family, KIS = KISIP lesson, DOC = GRM doctrine. Detail lives in the referenced spec.

GEN-CFG — Configuration & multi-tenancy (spec 01, 02)

ID	P	Requirement	Source
GEN-CFG-01	M	All tenant behavior (hierarchy, taxonomy, workflow, SLAs, forms, templates, roles, channels, branding) defined in a versioned DB configuration registry; no code change for routine variation	K2 NFR-14, KIS
GEN-CFG-02	M	Config versions validated (schema, references, workflow reachability) before activation; activation atomic	—
GEN-CFG-03	M	Config history with diff, author, note; rollback	K2 §4.6
GEN-CFG-04	M	Tenant profile export/import as signed bundle (env promotion, onboarding templates)	K2 NFR-17
GEN-CFG-05	M	Row-level tenant isolation in all data access (both deployment models)	—
GEN-CFG-06	M	In-flight cases pinned to their workflow version; explicit logged migration mapping	—
GEN-CFG-07	S	Dry-run/simulation of draft workflows	—
GEN-CFG-08	M	Per-tenant locales; all complainant-facing text localizable; EN+SW shipped	K2 FR-PUB-10/NFR-12
GEN-CFG-09	M	Feature flags per module (KB, tasks, committees, appeals, satisfaction, USSD, hotline, accounts, public API, transparency page)	—
GEN-CFG-10	M	Arbitrary-depth jurisdiction hierarchy (levels + unit tree, CSV import)	KIS vs K2 hierarchy mismatch

GEN-INT — Intake & channels (spec 05)

ID	P	Requirement	Source
GEN-INT-01	M	Unified register: all channels → standardized intake dataset, unique reference, channel tag	K2 §4.1
GEN-INT-02	M	Web portal: configured multi-step form, attachments, CAPTCHA, confirmation with reference	K2 FR-PUB-03/04/05/09
GEN-INT-03	M	Staff-assisted intake with source channel, scripts, draft save, party lookup-or-create	K2 FR-STAFF-04/05
GEN-INT-04	M	Anonymous submission (per-tenant policy) with reference-based tracking; never blocks logging or referral	K2 FR-PUB-13, §18.2
GEN-INT-05	M	Representative/group submissions with consent-of-affected-person	K2 Appendix A
GEN-INT-06	M	Consent capture (versioned privacy notice) whenever PII present	K2 FR-PUB-11, DPA
GEN-INT-07	M	Configurable duplicate detection (warn/link/queue-for-merge); no silent DB-constraint rejections	KIS
GEN-INT-08	M	Concurrency-safe configurable reference numbering	KIS (max()+1 bug class)
GEN-INT-09	M	Public status tracking via reference + verifier / signed link; enumeration-resistant; PII-minimized	K2 FR-PUB-06; KIS (guessable IDs)
GEN-INT-10	S	USSD intake (menu config, minimum fields, `requires_completion` triage) + status enquiry	K2 FR-USSD-01–04
GEN-INT-11	S	Hotline/IVR assisted intake with call metadata	K2 FR-HOT-01–05

ID	P	Requirement	Source
GEN-INT-12	S	Inbound email processing to triage queue; outbound email Must (see GEN-NOT)	K2 FR-EMAIL-02
GEN-INT-13	M	Chatbot intake default-off platform-wide; activates only by explicit tenant opt-in (see GEN-AI)	K2 FR-PUB-15
GEN-INT-14	M	Attachment policy enforcement (size/type/count) + authorized download with audit	K2 FR-ATT-01/02
GEN-INT-15	S	Malware scanning hook for uploads	K2 FR-ATT-03
GEN-INT-16	M	Free-of-charge, confidentiality and non-retaliation assurances displayed (content editable, presence enforced)	K2 FR-PUB-12/14, DOC
GEN-INT-17	S	Registered complainant accounts (OTP) with case history	K2 FR-PUB-07
GEN-INT-18	S	Knowledge base: categories, search, featured, multi-language; canned responses for staff	K2 FR-KB-01–03

GEN-AI — Chatbot & AI assistance (spec 05 §7) — all opt-in, default off

ID	P	Requirement	Source
GEN-AI-01	C	Chatbot channel adapter (web widget, WhatsApp): slot-filling over the configured form, read-back confirmation, KB-RAG FAQ, status check; transcript on case; automated-agent disclosure enforced	KIS document_ai precedent
GEN-AI-02	C	Mandatory human handoff path; automatic handoff on sensitivity signals	DOC survivor-centred
GEN-AI-03	C	AI triage suggestions: auto-categorization, priority, semantic dedupe — suggestion-only, staff confirm	—
GEN-AI-04	C	AI sensitivity detection: positive flag applies restrictions immediately, pending human confirmation; never auto-dismisses	—
GEN-AI-05	C	AI staff aids: summarization, translation (originals preserved), draft responses (edit/approve before send), KB answer assist with citations	—
GEN-AI-06	M*	AI governance (applies whenever any AI capability is on): human-in-the-loop for all case decisions, PII redaction before external calls, provider profiles with residency/no-training flags, full audit of every suggestion + accept/reject, per-capability kill switches, quality/drift dashboards	DPA 2019

* GEN-AI-06 is Must *conditional on* enabling any GEN-AI capability.

GEN-WF — Workflow, SLA, escalation, closure (spec 04)

ID	P	Requirement	Source
GEN-WF-01	M	Server-side state machine from config: statuses (with semantic tags), transitions, guards	KIS (client-side rules)
GEN-WF-02	M	Single atomic case-action endpoint (status+log+attachments+notifications outbox in one transaction; idempotency keys)	KIS (client rollback)
GEN-WF-03	M	Guards: role, jurisdiction level, required fields/attachments/notes, approvals, case state	K2 §4.2
GEN-WF-04	M	Screening step with eligibility outcome + out-of-scope referral guidance recorded	DOC, K2
GEN-WF-05	M	Escalation = level move up the configured tree; return = move down; direction from hierarchy config	KIS/K2
GEN-WF-06	M	SLA engine: plans (ack/first-response/resolution/stage), working calendars + holidays, server-computed clocks, pause statuses	K2 FR-ADM-06; KIS (ms bug)
GEN-WF-07	M	At-risk + breach detection on fixed scheduler; queue flags; reminders	K2 FR-STAFF-16/FR-NOTIF-03; KIS (random cron)
GEN-WF-08	M	Declarative escalation rules (time/severity/dissatisfaction/manual) with audited firings	K2 FR-WF-01
GEN-WF-09	M	Closure pipeline: resolution evidence, optional higher-level confirmation, satisfaction capture, appeal window, closure checklist + reason codes	KIS confirm pattern, K2 §4.4, DOC

ID	P	Requirement	Source
GEN-WF-10	M	Satisfaction capture (configurable channels; N/A for anonymous); dissatisfaction can trigger appeal	K2 FR-WF-02
GEN-WF-11	M	Appeal workflow: window, routing, rounds, decision capture; reopen policy with audit	K2 FR-WF-03, DOC
GEN-WF-12	M	External referrals: institution directory, structured handoff, external reference, outcome return	K2 §4.2, DOC
GEN-WF-13	M	Link related cases; merge with preserved threads and dual audit	K2 FR-STAFF-18/19
GEN-WF-14	M	Legal hold blocks closure-side mutations and retention	K2 §4.4
GEN-WF-15	S	Tasks module (create/link/assign/due/overdue alerts)	K2 FR-TASK-01–04
GEN-WF-16	S	Committee module: rosters, decision records bound to transitions	DOC (GRC/SAIC), KIS SEC/GRC
GEN-WF-17	M	Soft delete/archive with restore permission; permanent deletion only via retention jobs	KIS
GEN-WF-18	S	Complainant self-service actions per config (reply, appeal, self-escalate, withdraw) — verified, rate-limited	KIS self-escalate

GEN-CASE — Case management console (spec 03, 05)

ID	P	Requirement	Source
GEN-CASE-01	M	Queues: open/my/team/overdue/closed + saved views (filters, columns, sharing)	K2 FR-STAFF-02, ADM queues
GEN-CASE-02	M	Search: basic + advanced criteria builder, RBAC-scoped; sensitive cases excluded without clearance	K2 FR-STAFF-03; KIS
GEN-CASE-03	M	Case detail: timeline (single event stream), thread (external vs internal), parties, tasks, attachments, SLA clocks, available actions from engine	K2 FR-STAFF-07/08/17
GEN-CASE-04	M	Field edits audited with before/after; revert as new inverse event	KIS history/revert
GEN-CASE-05	M	Assignment/transfer with reason; collaborator management; claim-on-response option	K2 FR-STAFF-14/15/06
GEN-CASE-06	M	Bulk actions (assign/transfer/status where guards allow) with per-case guard evaluation + per-case audit	K2 FR-STAFF-21; KIS bulk refer
GEN-CASE-07	M	Case file PDF export (tenant-branded dossier)	KIS pdf endpoints, K2 FR-STAFF-20
GEN-CASE-08	M	Complainant directory (party history); org directory optional	K2 FR-USER-01, FR-ORG-01

GEN-NOT — Notifications (spec 06)

ID	P	Requirement	Source
GEN-NOT-01	M	Declarative event→recipient→template→channel rules; recipient selectors over roles+scopes	KIS (roleid 4 hardcode)
GEN-NOT-02	M	Multi-locale, multi-channel templates with validated variables; privacy-safe variants forced for sensitive classes	K2 FR-NOTIF-01/02
GEN-NOT-03	M	Transactional outbox; async delivery with retries; provider adapters (SMS/email/in-app)	—
GEN-NOT-04	M	Full notification log incl. suppressed sends with reason; failure dashboard + resend	KIS pattern, K2 INT-08/09
GEN-NOT-05	M	Per tenant×channel×module kill switches (audited)	KIS module_settings
GEN-NOT-06	S	Delivery callbacks, dedup/throttling, quiet hours	K2 INT-08

GEN-SEC — Security, access, PII (spec 07)

ID	P	Requirement	Source
GEN-SEC-01	M	AuthN on all non-public endpoints; public surface rate-limited + abuse-monitored	KIS (open writes)
GEN-SEC-02	M	RBAC: fixed permission catalogue composed into tenant roles; separation of admin vs handling	K2 AC-01, FR-ADM-01
GEN-SEC-03	M	Jurisdiction-subtree scoping enforced at data layer; record-level rules (assignee/dept)	K2 AC-02
GEN-SEC-04	M	Sensitivity classes with policy bundles (visibility, routing, masking, notification redaction, aggregate-only reporting, export block)	K2 FR-SENS-01–04; KIS isgbv
GEN-SEC-05	M	App-layer field encryption for PII (per-tenant keys, KMS envelope); HMAC lookup hashes	KIS pgp/AES defects
GEN-SEC-06	M	MFA configurable; mandatory for privileged roles; SSO (OIDC/SAML) per tenant	K2 AC-04/NFR-06/§16.5
GEN-SEC-07	M	Append-only hash-chained audit incl. sensitive-case views, exports, config changes, denials	K2 NFR-09
GEN-SEC-08	M	Retention policies (anonymize/archive/delete) per case type/sensitivity; legal hold; DSAR workflows	K2 §4.4, DPA 2019
GEN-SEC-09	M	Secrets in vault; no secrets in code/client; environment isolation; no prod PII in non-prod	KIS .env files
GEN-SEC-10	S	Break-glass access with justification + DPO alert	K2 §18.2
GEN-SEC-11	M	Time-bounded role assignments (valid_from/valid_to)	KIS role expiry
GEN-SEC-12	S	IP ACLs for staff/admin consoles; per-key API allowlists	K2 NFR-07

GEN-REP — Reporting (spec 08)

ID	P	Requirement	Source
GEN-REP-01	M	Standard KPI set on semantic tags + clocks; tenant targets; actual-vs-target dashboards	DOC KPIs, K2 REP-07
GEN-REP-02	M	Operational + management dashboards with tree drill-down; scope/sensitivity respected	K2 REP-01/02/03/06
GEN-REP-03	M	Exports (CSV/XLSX/PDF) gated by export field policies; PII excluded by default	K2 NFR-13, INT-10
GEN-REP-04	M	Aggregate-only sensitive reporting with minimum cell size	K2 FR-SENS-04
GEN-REP-05	S	Scheduled reports; regulator pack templates	K2 REP-05
GEN-REP-06	S	Public transparency page (tenant-approved aggregates)	ESS10/ATI
GEN-REP-07	S	Satisfaction & escalation analytics; repeat-complainant view	K2 REP-08
GEN-REP-08	C	Map dashboard over unit geo data	KIS geo data
GEN-REP-09	M	Admin-built dashboards as config: dashboard → sections → declarative widgets (dataset, measure, aggregation, group-by, filters, chart kind, targets, drill-down) with audience targeting and live-preview builder	KIS dashboard/section/card/chart system
GEN-REP-10	M	Widgets query only curated semantic-layer datasets with server-enforced jurisdiction + sensitivity row-level security, PII exclusion, cardinality/row caps and managed caching	KIS open summary endpoints (defect)
GEN-REP-11	M	Extensible chart-renderer catalogue (kpi card, bar/stacked, line/multi-line, pie/donut, treemap, map, table, pyramid); localized captions/source notes	KIS chart-types
GEN-REP-12	S	Default dashboard pack shipped as tenant-editable config; `is_main` per audience; dashboards portable via config bundle	KIS main_dashboard/public flags

GEN-API — API & integrations (spec 09)

ID	P	Requirement	Source
GEN-API-01	M	Documented OpenAPI REST surface; consoles are API clients	K2 NFR-16/INT-11
GEN-API-02	M	Idempotent mutations; optimistic concurrency; consistent errors	—
GEN-API-03	S	Outbound webhooks (signed, retried, PII-minimized)	K2 §16.3
GEN-API-04	M	Gateway adapters: SMS, USSD, email; provider-pluggable	K2 INT-06/07; KIS Advanta
GEN-API-05	M	Full data + config export in open formats (anti-lock-in)	K2 NFR-17
GEN-API-06	M	Validated, authenticated, logged bulk import jobs (units, users, legacy cases)	KIS /upsert

GEN-NFR — Non-functional

ID	P	Requirement	Source
GEN-NFR-01	M	95% of interactive requests < 3s; search < 5s at expected peak	K2 NFR-01
GEN-NFR-02	M	≥99.5% availability; maintenance page; documented scaling	K2 NFR-02/03
GEN-NFR-03	M	WCAG 2.1 AA; low-bandwidth public pages; responsive	K2 NFR-11
GEN-NFR-04	M	Deployment-model flexibility: gov-hosted dedicated or SaaS multi-tenant; data residency per contract	K2 24.4.2
GEN-NFR-05	M	Backups (daily full + incremental), tested restore, DR runbook with agreed RTO/RPO	K2 §5.2, DEL-13/14
GEN-NFR-06	M	Observability: structured logs, metrics, alerting (queue depth, notification failures, SLA scheduler health)	—
GEN-NFR-07	M	TLS everywhere; security headers; OWASP ASVS L2; CI dependency scanning	K2 NFR-04
GEN-NFR-08	S	Compliance report exports (filterable audit packs)	K2 NFR-19

Traceability

Every GEN-* requirement maps to: (a) the spec section that details it, (b) its source (KUSP2 ID family / KISIP lesson / doctrine), and (c) — during implementation — test cases in the RTM. The KUSP2 compliance matrix (136 IDs) can be answered from this catalogue almost one-to-one, which doubles as a bid-readiness artifact for similar procurements.

11 — Worked Tenant Profiles

Proof of genericity: the two known programmes expressed purely as configuration of the same platform. (Abbreviated — full bundles would enumerate every template and unit.)

1. Tenant profile: KISIP (informal settlements programme)

```yaml
tenant:
  code: kisip
  name: "Kenya Informal Settlements Improvement Project"
  locales: [en, sw]
  deployment: dedicated   # gov-hosted, own DB

hierarchy:
  levels: [settlement, county, national]      # rank 1..3
  flags:
    settlement: {is_intake_default: true}
    national:   {is_confirmation_authority: true}
  units: {import: kisip_units.csv}            # settlements, 47 counties, 1 national

taxonomy:
  case_types: [grievance, incident]           # incident = separate workflow (safety)
  categories: [land, evictions, compensation, infrastructure, drainage, water,
               sanitation, electricity, waste, environmental, health_safety,
               corruption, discrimination, gbv, labour, information_gap, delays, other]
  category_overrides:
    gbv:        {sensitivity: gbv_seah}
    corruption: {sensitivity: corruption}
  priorities: [normal, high, emergency]
  custom_fields:
    - {key: project_phase, type: select, list: [KISIP 1, KISIP 2], default: KISIP 2}
    - {key: in_court, type: boolean}

workflow: # statuses & transitions as spec 02 §4 example
  initial:
    default: Sorting
    rules:
      - {if: {field: in_court}, then: In Court}
      - {if: {sensitivity: gbv_seah}, then: {status: Sorting, level: national}}
  closure:
    confirmation: {required_when: {resolved_below: national}, authority_level: national}
    satisfaction: {enabled: true, channels: [sms]}
  appeal: {enabled: true, window_days: 30, routes_to: next_level}

sla:
  plans:
    standard: {stage: {Sorting: 7d, Investigation: 14d, Escalated: 14d, Resolved: 21d}}
  calendar: kenya_holidays
  escalation_rules: [overdue-auto-escalate, dissatisfaction-appeal]

channels:
  web: {enabled: true, captcha: true, anonymous: allowed_via_toggle}
  assisted: {enabled: true}
  sms: {enabled: true, provider: advanta, sender_id: KISIP}
  ussd: {enabled: false}
  hotline: {enabled: false}
  chatbot: {enabled: false}

ai_assistance:   # KISIP already runs a document-AI service; formalized under CD-16 governance
  enabled: true
  provider: {kind: openai, profile: kisip_openai, pii_redaction: true}
  capabilities: {kb_answer_assist: true, summarization: true, auto_categorization: false}

numbering: {pattern: "GRM-{YYYY}-{seq:4}", scope: yearly}

notifications:
  rules: default_pack
  overrides:
    - {on: case.status_changed, condition: {to_status: Referred}, to: [], note: "no complainant SMS on internal referral"}
```

```
roles:
  grm_officer:        {permissions: [case:*, thread:*, attachment:*], sensitive: []}
  gbv_officer:        {inherits: grm_officer, sensitive: [gbv_seah]}
  county_admin:       {inherits: grm_officer, extra: [report:view_operational]}
  national_grm:       {inherits: grm_officer, extra: [case:confirm_resolution]}
  # scoping done per assignment: role × unit (settlement/county/national subtree)

committees:
  enabled: true
  defs: [{level: settlement, name: SEC}, {level: settlement, name: GRC}]
  workflow_bindings: []   # rosters only (current KISIP practice)
```

Migration notes (from `plus-admin`): map statuses 1:1 (they keep their names); legacy `grievance_log` + `grievance_history` → `case_event` stream; `grievance_notification` → `notification_log`; `isgbv` → sensitivity class; settlement/county/ward/subcounty FKs → unit tree references; encrypted name/national_id re-encrypted under app-layer keys; legacy `GRM-YYYY-####` references preserved.

2. Tenant profile: KUSP2 (urban programme)

```
tenant:
  code: kusp2
  name: "Second Kenya Urban Support Programme"
  locales: [en, sw]
  deployment: dedicated   # gov-hosted preferred (Appendix C Option A)

hierarchy:
  levels: [municipality, county, national]
  flags:
    municipality: {is_intake_default: true}
    national: {is_confirmation_authority: false}   # KUSP2 uses supervisory approval instead
  units: {import: kusp2_units.csv}                 # 79 municipalities, 45 counties, NPCT

taxonomy:
  case_types: [grievance]
  categories: [environmental, social, gbv_seah, corruption_fraud,
               project_implementation, institutional_policy, other]
  category_overrides:
    gbv_seah:         {sensitivity: gbv_seah}
    corruption_fraud: {sensitivity: corruption}
  priorities: [low, normal, high, emergency]
  expected_outcomes: [information, corrective_action, compensation, accountability, other]

workflow:
  statuses:
    - {name: Open/Pending,                tag: open}
    - {name: Awaiting Complainant Info,   tag: on_hold, pauses: [resolution, stage]}
    - {name: Under Investigation,         tag: in_progress}
    - {name: Action in Progress,          tag: in_progress}
    - {name: Escalated,                   tag: in_progress}   # level shown via current_level
    - {name: Referred,                    tag: in_progress}
    - {name: Resolved,                    tag: resolved}
    - {name: Closed,                      tag: closed}
  closure:
    confirmation: {required_when: never}
    supervisory_approval: {when: {priority_gte: high}, or: {sensitivity: any_nonstandard}, or: {escalated: true}}
    checklist: [resolution_summary, complainant_informed, satisfaction_captured]
    satisfaction: {enabled: true, channels: [sms, portal, staff_capture]}
  appeal: {enabled: true, routes_to: next_level, max_rounds: 2}

sla:
  plans:
    standard:
      acknowledge: immediate          # auto-ack on intake
      first_response: 14d_working
      resolve: 30d_working
  calendar: kenya_holidays
  reminders: [{at: T-2d, to: assignee}]
  escalation_rules: [overdue-auto-escalate, dissatisfaction-appeal, emergency-priority]

channels:
  web: {enabled: true, captcha: true, anonymous: allowed, accounts: optional}
  assisted: {enabled: true, scripts: kusp2_intake_scripts}
  email: {outbound: true, inbound: true}
  sms: {enabled: when_gateway_ready}
  ussd: {enabled: when_gateway_ready, min_fields: [unit, summary, category]}
  hotline: {enabled: when_gateway_ready}
  chatbot: {enabled: false}            # FR-PUB-15 prohibits chatbot intake

ai_assistance: {enabled: false}        # CD-16 stays off for KUSP2

numbering: {pattern: "{unit_code}-{YYYY}-{seq:5}", scope: per_unit_yearly}
```

```
roles:   # six training tracks → role templates
  administrator: {permissions: [admin:*]}
  supervisor:    {permissions: [case:*, report:*, appeal:decide]}
  grm_handler:   {permissions: [case:read, case:transition, thread:*, task:*]}
  intake_agent:  {permissions: [case:create_assisted, case:read]}
  me_analyst:    {permissions: [report:*], pii: none}
  seash_focal:   {inherits: grm_handler, sensitive: [gbv_seah]}

referral_directory: [CAJ_Ombudsman, EACC, NEMA, gbv_service_providers, world_bank_grs]
reporting:
  kpi_targets: {first_response_days: 14, resolution_days: 30}
  transparency_page: {enabled: true, aggregates: [received, resolved, avg_resolution_days, by_county]}
```

3. What the two profiles demonstrate

Variation	KISIP	KUSP2	Platform mechanism
Hierarchy	settlement/county/national	municipality/county/national	CD-02 unit tree
Closure control	national confirmation	supervisory approval + checklist	CD-04 closure policy
SLA style	per-stage durations	ack/response/resolution targets	CD-05 plan types
Primary channel	SMS-first	web/email-first, telecom staged	CD-08 + CD-09
Referral notice to complainant	suppressed	standard	notification rule condition
Numbering	global yearly `GRM-YYYY-####`	per-unit yearly	CD-07 pattern
Sensitive routing	GBV → national	GBV → SEA/SH focals at any level	sensitivity class policy
Committees	rosters only	not used (departments/teams)	CD-11 feature flag

A third hypothetical client (corporate external GRM à la Cassava: 2-day ack, 30-day resolve, GRC committee escalation, anonymous hotline) is expressible with the same domains — no new code paths.

12 — Development Plan (Nuxt UI Pro + Node.js)

How to build the platform specified in docs 01–11. Stack: **Nuxt 4 + Nuxt UI Pro** on the frontend, **Node.js (TypeScript)** services on the backend, **PostgreSQL** as the system of record. Scope baseline = all **Must (M)** rows in the requirements catalogue (doc 10); Should/Could rows are scheduled as fast-follows.

1. Stack decisions

Layer	Choice	Rationale
Frontend framework	**Nuxt 4** (Vue 3, TypeScript)	Team continuity with the Vue-based KISIP codebase; SSR for the public portal (SEO, low-bandwidth, WCAG); SPA-mode app shell for consoles
UI kit	**Nuxt UI Pro**	Production-ready dashboard/page/form components (DashboardLayout, Tables, CommandPalette, Forms, PageSections) cover ~80% of console & portal chrome; Tailwind-based theming maps directly onto CD-01 tenant branding
State / data	Pinia + Nuxt `useFetch` /typed SDK	Generated OpenAPI client keeps consoles honest API clients (GEN-API-01)
i18n	`@nuxtjs/i18n`	EN+SW shipped; per-tenant locales (GEN-CFG-08)
Charts	Apex/ECharts wrappers	Reuse the proven KISIP chart-kind catalogue for the dashboard renderer registry (GEN-REP-11)
API service	**Node.js 22 + Fastify + TypeScript**	Lightweight, fast, first-class JSON-schema validation (pairs with config-registry schema validation), OpenAPI generation via `fastify-swagger`
ORM / DB	**Drizzle ORM + PostgreSQL 16**	Typed schema-as-code migrations; row-level tenant isolation (GEN-CFG-05); `pgcrypto` -free app-layer PII encryption (GEN-SEC-05)
Jobs / outbox	**BullMQ + Redis**	Notification outbox dispatch, SLA scheduler ticks, retention jobs, report generation (GEN-NOT-03, GEN-WF-07)
Auth	OIDC (Keycloak or Auth.js + node-oidc-provider), JWT access + refresh	SSO per tenant, MFA, session policy (GEN-SEC-06)
Files	S3-compatible object store (MinIO on-prem)	Attachment policy + scanning hook (GEN-INT-14/15)
AI (opt-in)	Provider-adapter package (OpenAI/Azure/xAI/local via Ollama)	CD-16 provider profiles; PII redaction middleware before egress
Infra	Docker Compose (dev) → Kubernetes or Docker Swarm (prod); GitHub Actions CI	Gov-hosted dedicated or SaaS multi-tenant (GEN-NFR-04)

Architecture shape (from doc 01): the Nuxt apps are pure API clients of one versioned REST surface. Nuxt's Nitro server is used only for SSR/edge caching of the public portal — no business logic in the frontend tier. The workflow engine, SLA clocks and notification rules live exclusively in the API/worker services (platform invariant: *the server owns the rules*).

2. Monorepo layout

```
egrm/
├── apps/
│   ├── portal/        # Nuxt 4 + UI Pro — public portal (SSR): submit, track, KB, transparency page
│   ├── console/       # Nuxt 4 + UI Pro — staff console + admin console (SPA mode, route-split)
│   ├── api/           # Fastify — REST API, workflow engine, config registry, semantic layer
│   └── worker/        # BullMQ consumers — outbox dispatch, SLA scheduler, retention, reports, AI calls
├── packages/
│   ├── config-schemas/  # zod/JSON-schema for all CD-01…CD-16 config domains (shared validation)
│   ├── sdk/             # OpenAPI-generated typed client (used by both Nuxt apps)
│   ├── ui/              # shared Nuxt UI Pro theme extensions, tenant theming, chart renderers
│   └── core/            # shared domain types, semantic status tags, permission catalogue constants
├── infra/               # docker-compose, k8s manifests, CI workflows
└── specs/               # these documents (source of truth)
```

pnpm workspaces + Turborepo. One deployable image per app; tenant resolution by hostname (SaaS) or build-time env (dedicated instance).

3. Delivery phases

Each phase ends with a demoable increment, migration scripts, and tests green in CI. Requirement IDs reference doc 10.

Phase 0 — Foundations (weeks 1–4)

- Monorepo, CI/CD (lint, typecheck, unit + API contract tests, preview deploys), Docker dev stack (Postgres, Redis, MinIO, Mailpit).
- **Tenancy kernel**: tenant table, row-level isolation middleware, hostname resolution (GEN-CFG-05).
- **Auth**: OIDC login, JWT guards, password policy, session management; user/role tables from the fixed permission catalogue (GEN-SEC-01/02).
- **Config registry v1**: versioned config storage, `config-schemas` package, validation-before-activation, audit events (GEN-CFG-01/02/03).
- Nuxt apps scaffolded with UI Pro: authenticated console shell (nav from permissions), portal shell with tenant theming (CD-01).
- *Exit criteria*: log in to an empty console of a seeded tenant whose branding/locales come entirely from config.

Phase 1 — Domain core & intake (weeks 5–10)

- Domain schema: case, party (PII-encrypted columns + HMAC lookup hashes), case_event stream, attachments, jurisdiction hierarchy (arbitrary depth + CSV import) (GEN-CFG-10, GEN-SEC-05).
- **Intake pipeline**: web portal multi-step form rendered from CD-06 form definitions; assisted intake in console; consent capture; dedupe rule; concurrency-safe reference numbering (GEN-INT-01…08).
- Public **status tracking** (reference + verifier, enumeration-resistant) (GEN-INT-09).
- Case detail page: timeline, thread (external/internal), attachments (GEN-CASE-03), queues + saved views (GEN-CASE-01).
- *Exit criteria*: a grievance submitted on the portal appears in the console queue, trackable publicly; the same build serves a second tenant with a different hierarchy and form.

Phase 2 — Workflow engine, SLA & notifications (weeks 11–17)

- **Server-side state machine** interpreting CD-04 workflow config: statuses, semantic tags, transitions with guards; single atomic case-action endpoint with idempotency keys (GEN-WF-01/02/03).
- **SLA engine**: plans, working calendars, server-computed clocks, at-risk/breach scheduler in worker (fixes KISIP's days-vs-ms bug class) (GEN-WF-06/07); declarative escalation rules (GEN-WF-08).
- **Notification engine**: event→recipient→template→channel rules, transactional outbox, provider adapters (SMS: Advanta + pluggable; email: SMTP), full send/suppress log, kill switches (GEN-NOT-01…05).
- Closure pipeline: resolution evidence, confirmation step, satisfaction capture, appeal window (GEN-WF-09/10/11); merge/link, referrals, legal hold (GEN-WF-12/13/14).
- *Exit criteria*: both KISIP and KUSP2 workflow profiles (doc 11) run end-to-end — intake → escalate → resolve → confirm → close → appeal — on one build, with SMS/email firing per rules.

Phase 3 — Security hardening & sensitive cases (weeks 18–21)

- Sensitivity classes: visibility restriction, designated-role routing, notification redaction, aggregate-only reporting (GEN-SEC-04).
- Jurisdiction-subtree scoping at the data layer; field-edit audit with before/after; append-only hash-chained audit log (GEN-SEC-03/07, GEN-CASE-04).
- MFA, retention jobs (anonymize/archive/delete), DSAR support, secrets vault integration (GEN-SEC-06/08/09).
- External pen test + OWASP ASVS L2 review (GEN-NFR-07).
- *Exit criteria*: GBV-class case invisible to non-designated staff everywhere (queues, search, reports, notifications); audit pack export.

Phase 4 — Reporting & configurable dashboards (weeks 22–26)

- **Semantic layer**: curated datasets over the event stream with mandatory row-level security, materialized aggregates in worker (GEN-REP-10).
- **Dashboard builder**: dashboard → section → widget config (CD-15), live-preview editor in admin console, chart-renderer registry in `packages/ui` (port of the KISIP chart kinds), default dashboard pack (GEN-REP-09/11/12).
- Standard KPI set + targets; exports with field policies; case-file PDF dossier; transparency page (GEN-REP-01…04, GEN-CASE-07).
- *Exit criteria*: admin builds a new chart from the console with no deploy; KPIs match hand-computed values on seed data.

Phase 5 — Extended channels & integrations (weeks 27–31)

- USSD adapter (menu tree from config) + status enquiry; hotline intake metadata; inbound email triage queue (GEN-INT-10/11/12).
- Public API + webhooks (signed, retried); bulk import jobs (units, users, legacy KISIP cases); full data/config export (GEN-API-03…06).
- Knowledge base + canned responses; registered complainant accounts (GEN-INT-17/18).
- *Exit criteria*: KISIP legacy data imported into a tenant; webhook consumer demo; USSD sandbox flow.

Phase 6 — Chatbot & AI assistance, opt-in (weeks 32–36)

- AI provider-adapter package with PII-redaction middleware, `ai_interaction` audit, kill switches, quality dashboard (GEN-AI-06 governance first).
- Staff aids: auto-categorization, semantic dedupe, summarization, draft responses, KB answer assist (GEN-AI-03/05); sensitivity detection with restriction-first behavior (GEN-AI-04).
- Chatbot channel adapter (web widget; WhatsApp behind gateway), slot-filling over form config, human handoff (GEN-AI-01/02).
- *Exit criteria*: AI suggestions visibly improve triage on the pilot tenant with 100% of decisions human-confirmed and audited.

Phase 7 — Pilot, UAT & production (weeks 35–40, overlaps 6)

- Performance/load testing to NFR targets (95% < 3s at 100k cases), DR drill (backup-restore), observability dashboards (GEN-NFR-01…06).
- Tenant onboarding runbook: author profile → import units/users → train → go live. Pilot with one tenant (KISIP migration or KUSP2 greenfield), 2-week hypercare.
- Documentation set: admin guide, API reference (generated), data dictionary, training materials per role track.

4. Team & timeline

Role	Allocation
Tech lead / architect	1 FTE, whole programme
Backend (Node/Fastify)	2 FTE
Frontend (Nuxt/UI Pro)	2 FTE
QA / test automation	1 FTE from Phase 1
DevOps	0.5 FTE
UX (portal accessibility, USSD/chatbot scripts)	0.5 FTE, Phases 0–2 and 6

≈ **40 weeks (~9 months)** to production pilot with this team; MVP demo (end of Phase 2, the full grievance lifecycle) at ~4 months. Compressible to ~30 weeks by running Phase 4 parallel to Phase 3 with a third backend dev.

5. Engineering practices

- **Spec traceability**: every PR references GEN-* IDs; the requirements catalogue becomes the test-plan backbone (RTM).
- **Testing pyramid**: unit (engine guards, SLA math, config validation), API contract tests against the OpenAPI schema, Playwright E2E for the four golden paths (submit→close, escalate, sensitive case, appeal), seeded multi-tenant fixtures (both doc-11 profiles run in CI).
- **Config-first development**: features are built against the two reference tenant profiles simultaneously — if a feature needs tenant-specific code, the design is wrong.
- **Migration discipline**: Drizzle migrations forward-only; config versions pinned to in-flight cases (GEN-CFG-06).
- **Definition of done**: code + tests + audit events + i18n keys (EN/SW) + permission checks + docs updated.

6. Key risks

Risk	Mitigation
Workflow-engine complexity creep	Engine interprets a constrained schema (doc 02 §4) — no scripting/expression language in v1; escape hatch = new guard types added by platform team
SMS/USSD gateway dependencies	Adapter contracts + sandbox simulators from Phase 2; telecom integration on the critical path only in Phase 5
PII encryption vs search/reporting	HMAC lookup hashes + semantic-layer design settled in Phase 1 schema, load-tested early
Nuxt UI Pro licensing	One-time license per developer; budget line item; components are owned code once built
AI governance acceptance (gov tenants)	AI entirely behind flags; local-model option (Ollama) for data-sovereign deployments
Scope pull from KUSP2 procurement timeline	Must-rows only until Phase 7; doc 10 priorities are the change-control baseline